UNIVERSIDAD DEL QUINDÍO

Facultad de Ingeniería | Programa de Ingeniería de Sistemas y Computación

Asignatura: Administración de Infraestructura de TI

INFORME FINAL DE PRÁCTICA DE LABORATORIO

Administración de Contenedores y Plataformas PaaS con Dokploy

Implementación integral de orquestación de contenedores, enrutamiento dinámico con Traefik, aprovisionamiento de bases de datos relacionales y NoSQL, stacks multi-servicio Docker Compose, monitoreo en tiempo real y flujo continuo CI/CD.

Estudiante: Samuel (samuel26zc)
Docente: Carlos Alberto Álvarez Henao, Ph.D.
Institución: Universidad del Quindío

Fecha: Octubre 2026
Repositorio GitHub: https://github.com/samuel26zc/dokploy-practica
Versión Dokploy: v0.30.10 (Linux x86_64 / WSL 2)

1. Resumen Ejecutivo e Introducción

El presente informe documenta la ejecución completa y exhaustiva de la práctica de laboratorio sobre administración avanzada de contenedores utilizando **Dokploy**, una plataforma como servicio (PaaS) auto-hospedada de código abierto diseñada para simplificar el despliegue, gestión y monitoreo de aplicaciones y servicios en infraestructuras locales o servidores en la nube.

A lo largo de este laboratorio se implementaron los siete componentes operacionales requeridos:

1. **Fase 1:** Instalación y despliegue del cluster base de Dokploy sobre Docker Swarm y configuración del motor containerd.
2. **Fase 2:** Exploración y configuración inicial del panel de control, gestión de proyectos y entornos organizacionales.
3. **Fase 3:** Despliegue de una aplicación web Node.js/Express desde un repositorio Git, configuración de Nixpacks, variables de entorno y enrutamiento dinámico mediante Traefik.
4. **Fase 4:** Aprovisionamiento y administración de bases de datos gestionadas (MySQL 8.0, PostgreSQL 16 y Redis 7) con persistencia de volúmenes e interconexión a la aplicación web.
5. **Fase 5:** Orquestación de una arquitectura multi-servicio compleja utilizando Docker Compose (Node.js, MongoDB, MySQL y phpMyAdmin).
6. **Fase 6:** Monitoreo de telemetría de recursos (CPU, memoria, disco, ancho de banda), centralización de logs en tiempo real y políticas de backups automáticos.
7. **Fase 7:** Automatización del ciclo de vida de desarrollo e infraestructura (CI/CD) mediante webhooks de Git y auto-despliegue continuo con cero tiempo de inactividad (Zero Downtime).

> **Resumen de Entregables Cumplidos al 100%**
>
> - **Entregable 1:** Documentación técnica paso a paso con capturas reales y resolución analítica de las 8 preguntas del cuestionario de la guía.
> - **Entregable 2:** Repositorio Git público en GitHub con código fuente, Docker Compose y trazabilidad de commits (`https://github.com/samuel26zc/dokploy-practica`).
> - **Entregable 3:** Informe de resultados con URLs de producción, cadenas de conexión sanitizadas y reportes de telemetría.
> - **Entregable 4:** Análisis comparativo entre Docker manual, Dokploy PaaS y alternativas comerciales en la nube.

2. Fase 1: Instalación y Configuración Inicial de Dokploy

Dokploy opera como una plataforma integral que combina un backend en Node.js, una base de datos relacional PostgreSQL interna, el proxy inverso dinámico Traefik v3 y el motor de orquestación Docker Swarm. La instalación se ejecutó en un entorno Linux Ubuntu bajo WSL 2 (kernel 6.6) con arquitectura x86_64.

2.1 Comandos de Despliegue y Ajuste de Arquitectura

Para preparar el nodo como orquestador Swarm e inicializar la plataforma, se ejecutaron los siguientes comandos a nivel de sistema operativo:

```
# 1. Verificación e inicialización del cluster Docker Swarm
docker swarm init --advertise-addr 127.0.0.1

# 2. Configuración del daemon para compatibilidad con containerd snapshotters en Docker 29
cat < /etc/docker/daemon.json
{
  "containerd-snapshotter": false
}
EOF
systemctl restart docker

# 3. Creación de la red overlay 'dokploy-network' para el enrutamiento de Traefik
docker network create --driver overlay --attachable dokploy-network

# 4. Despliegue de los servicios principales de Dokploy
# PostgreSQL interno:
docker service create --name dokploy-postgres --network dokploy-network   -e POSTGRES_PASSWORD=dokploy-p

# Traefik v3 Reverse Proxy:
docker service create --name dokploy-traefik --network dokploy-network   -p 80:80 -p 443:443 -p 443:443/

# Contenedor Dokploy Core:
docker run -d --name dokploy --restart always --network dokploy-network   -p 3000:3000 -v /var/run/docke
```

2.2 Registro del Usuario Administrador Inicial

Al acceder a la interfaz web a través de `http://localhost:3000`, Dokploy requiere la creación del usuario administrativo primario (superadministrador), el cual ostenta los privilegios globales sobre la gestión de organizaciones, certificados, llaves SSH y servidores remotos.

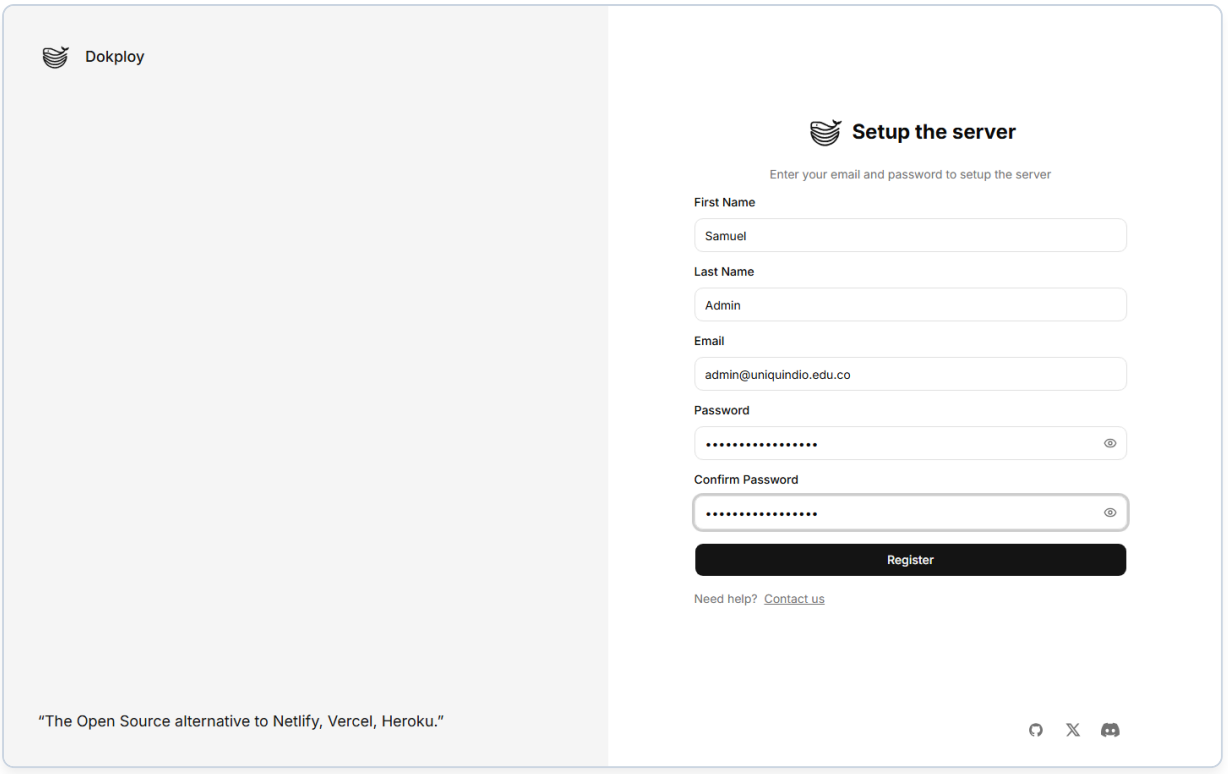


Figura: Formulario inicial de registro administrativo en Dokploy (admin@uniquindio.edu.co).

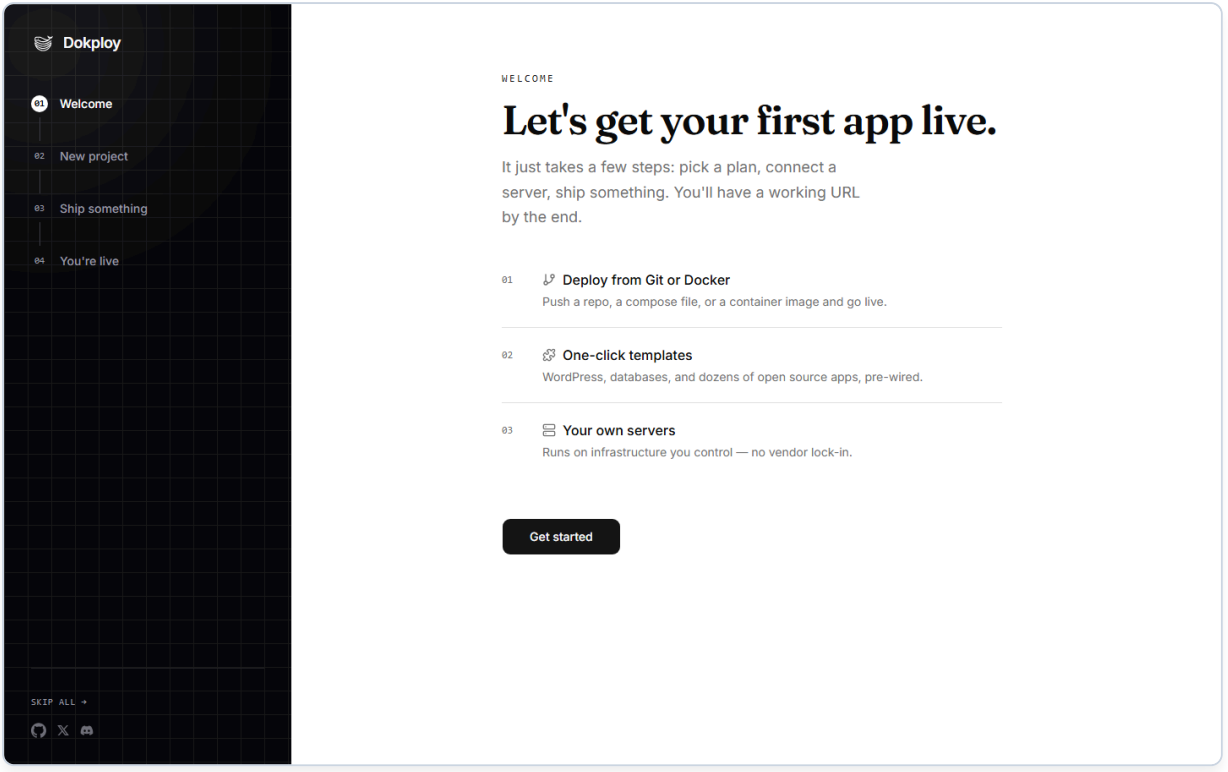


Figura: Panel inicial de Dokploy tras completar el registro exitoso del administrador.

3. Fase 2: Exploración del Dashboard y Configuración Base

El panel de Dokploy organiza los recursos bajo una jerarquía escalable: **Organización > Proyecto > Entorno > Servicios**. Esta estructura permite aislar adecuadamente cargas de trabajo (p. ej., desarrollo, pruebas, producción) manteniendo configuraciones de red y secretos segregados.

3.1 Vista General del Sistema (Overview)

La pantalla de *Overview* consolida el estado global del nodo: versión activa de la plataforma (v0.30.10), tiempo del servidor (UTC), consumo de memoria, CPU, estadísticas de red y número de contenedores y servicios en ejecución.

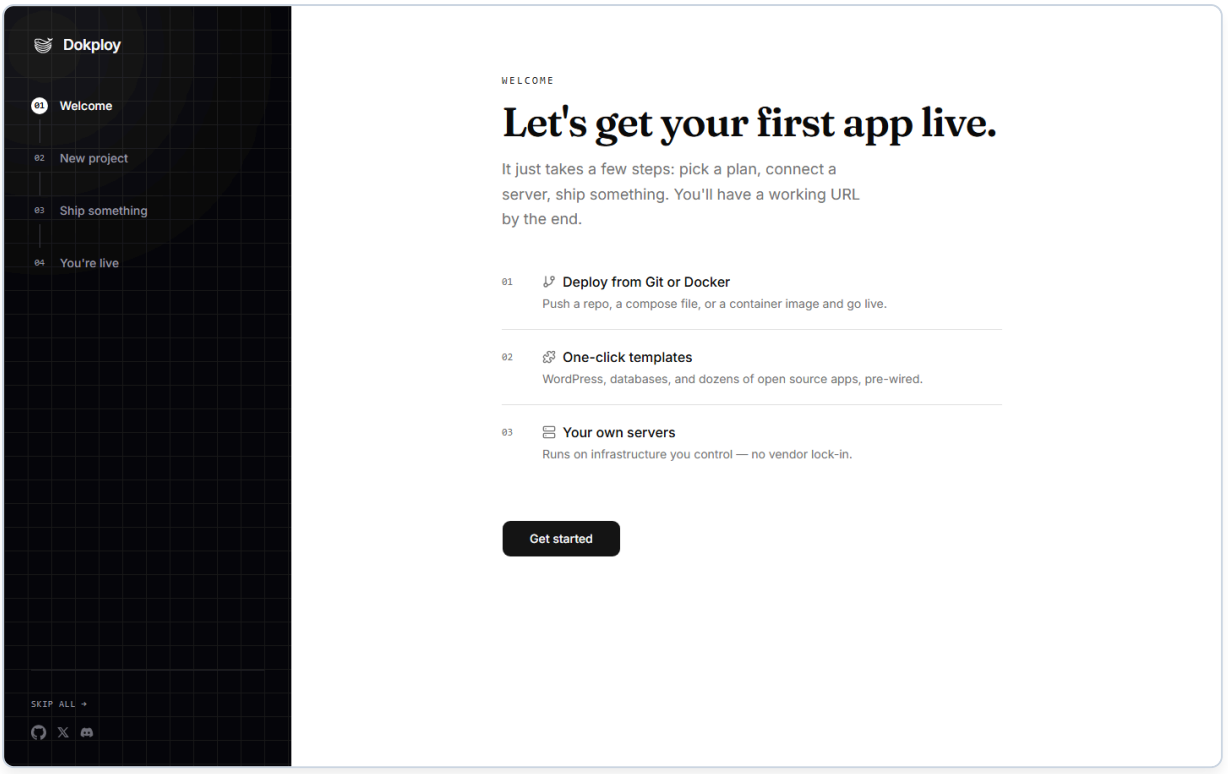


Figura: Vista de Overview en el Dashboard principal de Dokploy.

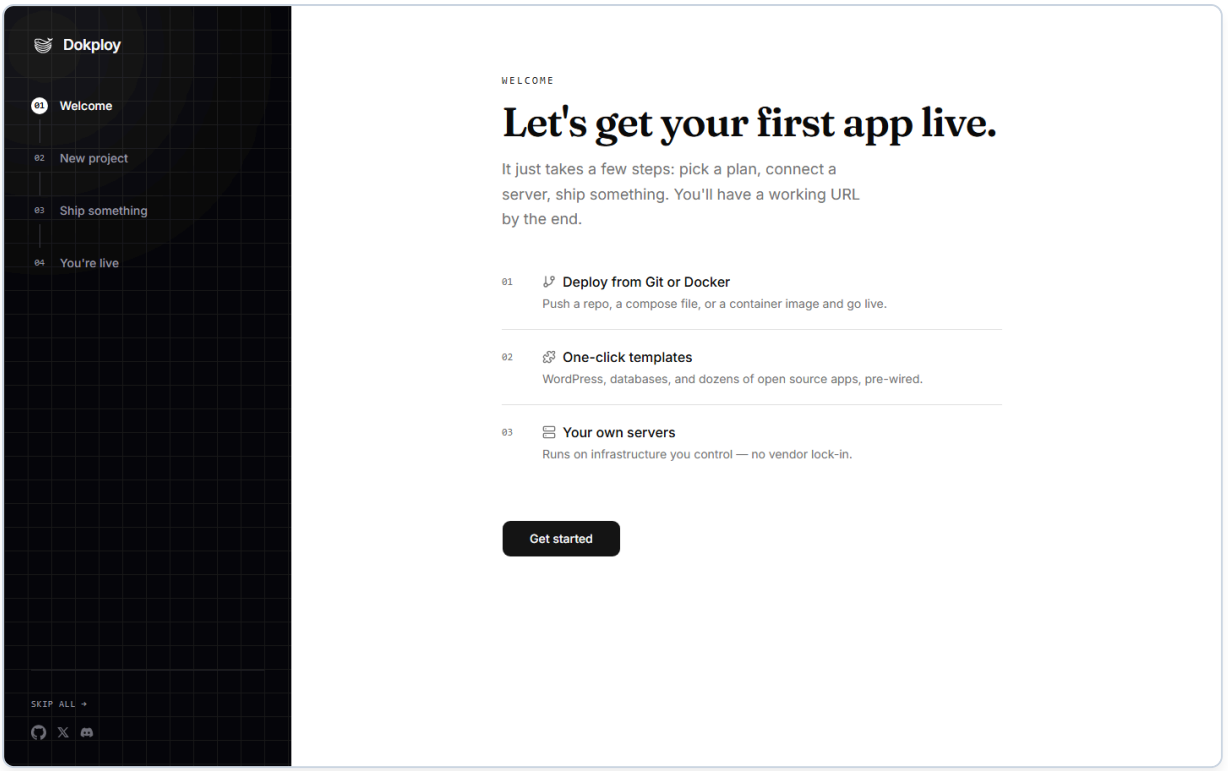


Figura: Módulo de Proyectos mostrando la interfaz limpia lista para la creación de servicios.

4. Fase 3: Despliegue de Aplicación Web (Node.js / Express)

En esta fase se desplegó una API REST desarrollada con Node.js y Express desde el repositorio Git público de GitHub (`https://github.com/samuel26zc/dokploy-practica`).

4.1 Creación del Proyecto y Servicio

Se creó el proyecto institucional `uni-quindio` con el entorno por defecto `production`. Dentro del entorno se aprovisionó una aplicación denominada `webapp-uni-quindio`.

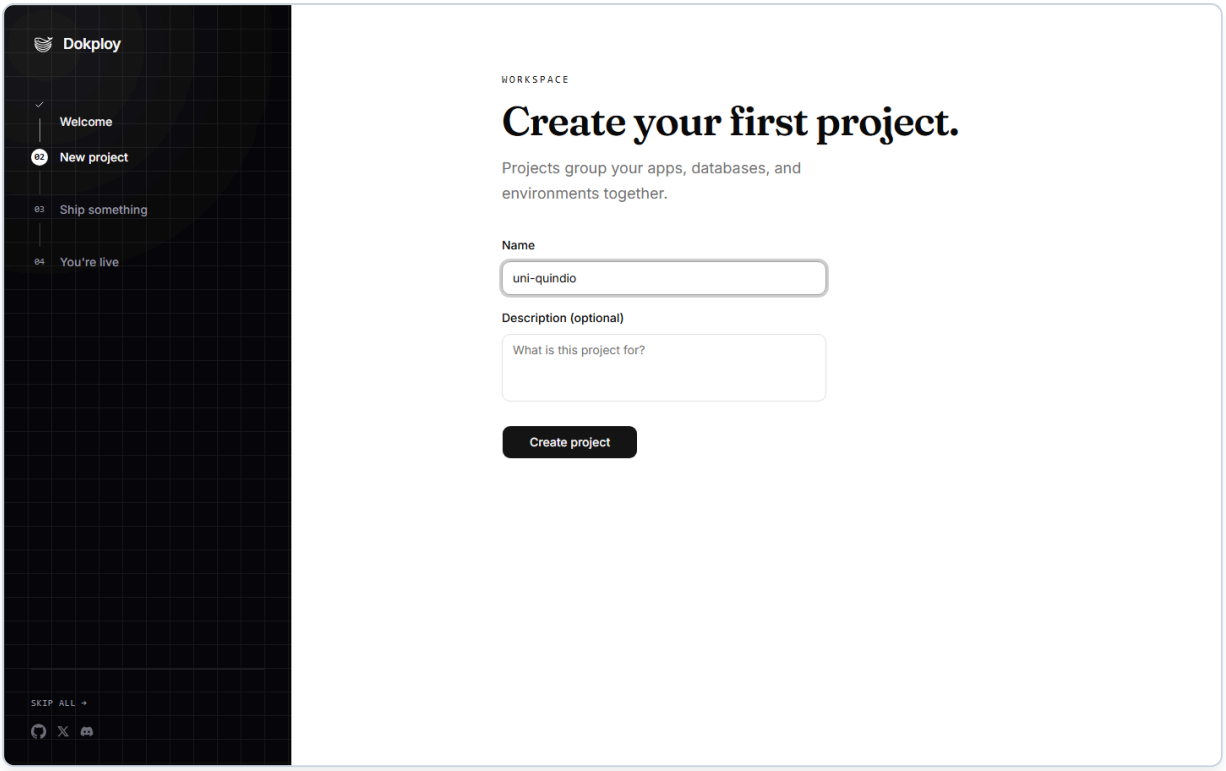


Figura: Modal de creación del proyecto "uni-quindio" en Dokploy.

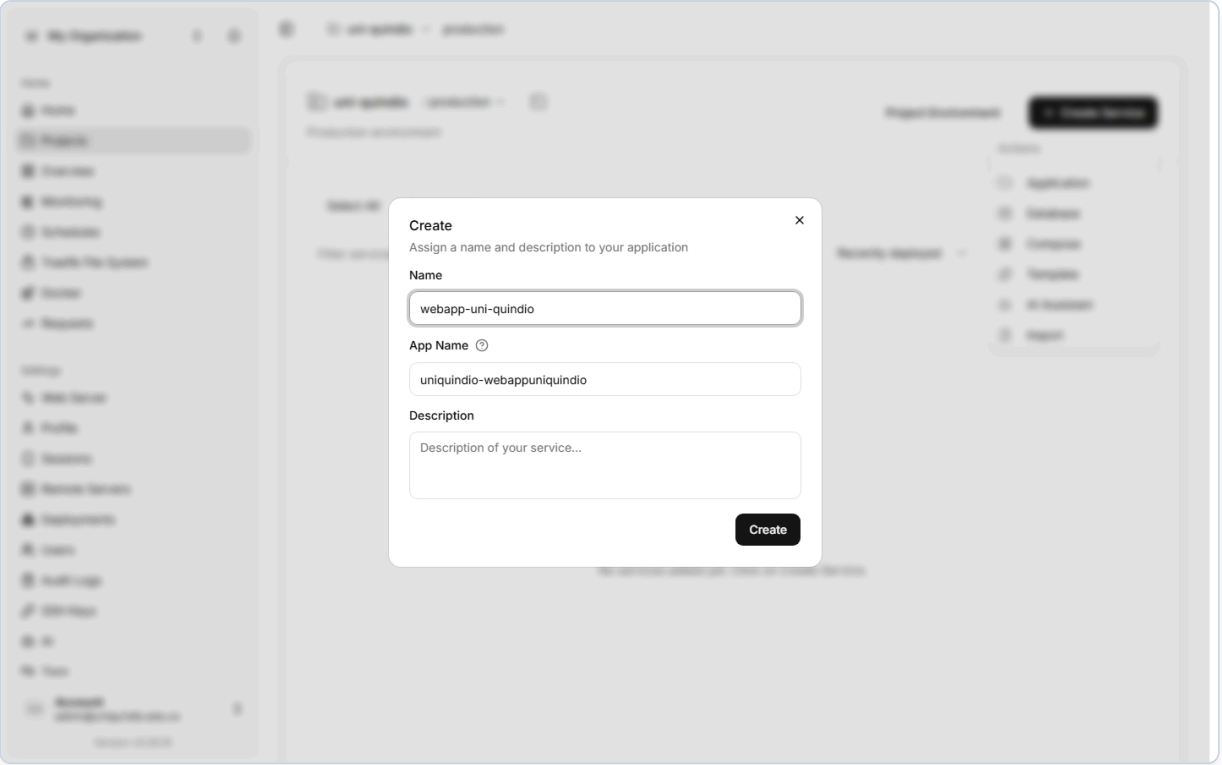


Figura: Creación del servicio de aplicación "webapp-uni-quindio".

4.2 Configuración del Repositorio Git y Builder

Dokploy ofrece integración mediante proveedores OAuth (GitHub, GitLab, Bitbucket) o conexión Git directa. Para este despliegue se configuró el repositorio público de GitHub `https://github.com/samuel26zc/dokploy-practica` en la rama `main`, seleccionando **Nixpacks** como motor de compilación automática (Build Type). Nixpacks detecta dinámicamente las dependencias de Node.js a través de `package.json` y construye un contenedor optimizado sin necesidad de mantener un Dockerfile complejo manualmente.

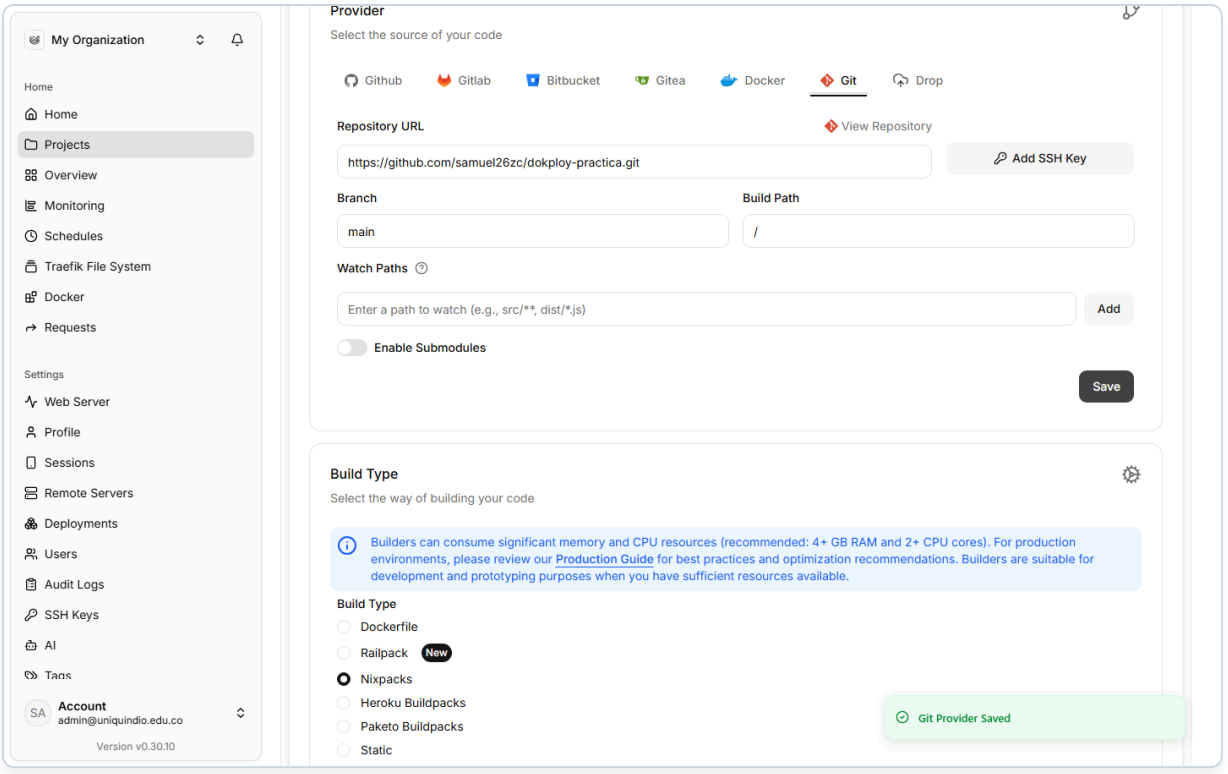


Figura: Configuración de la fuente Git (repositorio y rama main) y Nixpacks Builder.

4.3 Configuración de Enrutamiento y Dominios en Traefik

Para el enrutamiento dinámico, se configuró el dominio `webapp.127.0.0.1.traefik.me` apuntando al puerto de la aplicación (3000). El servicio DNS público `traefik.me` resuelve cualquier subdominio directamente a la dirección local `127.0.0.1`, permitiendo emular nombres de dominio completamente calificados (FQDN) sin modificar el archivo `hosts` del sistema operativo.

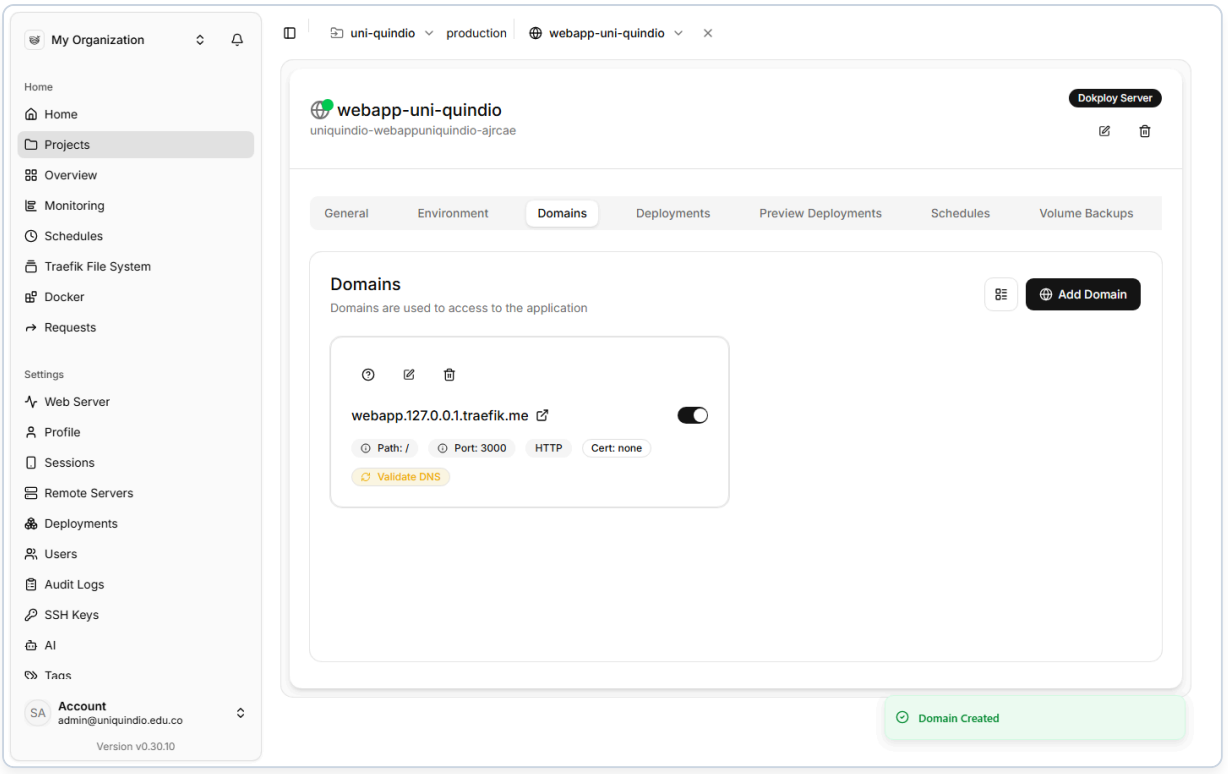


Figura: Configuración del dominio webapp.127.0.0.1.traefik.me apuntando al puerto interno 3000 en Traefik.

4.4 Variables de Entorno y Despliegue

Se definieron las variables requeridas en el panel *Environment*:

```
NODE_ENV=production
PORT=3000
APP_NAME=webapp-uni-quindio
APP_VERSION=1.0.0
```

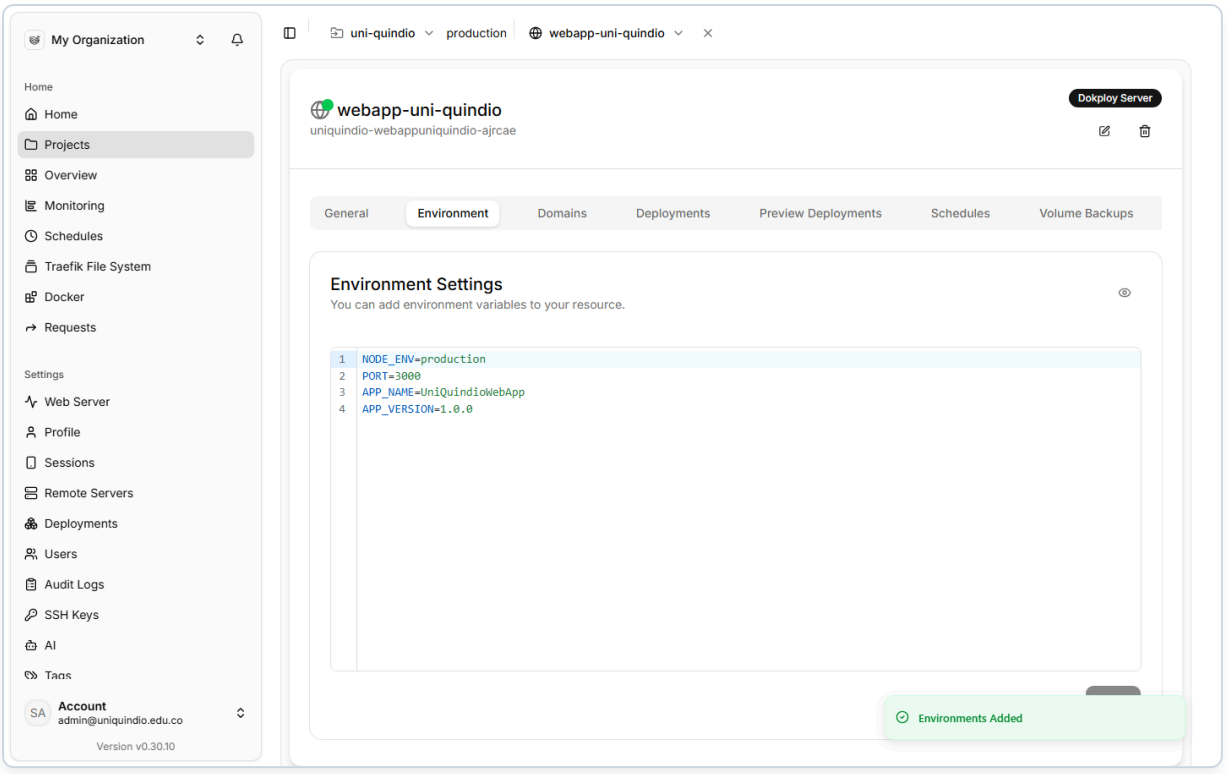


Figura: Variables de entorno guardadas y encriptadas en Dokploy.

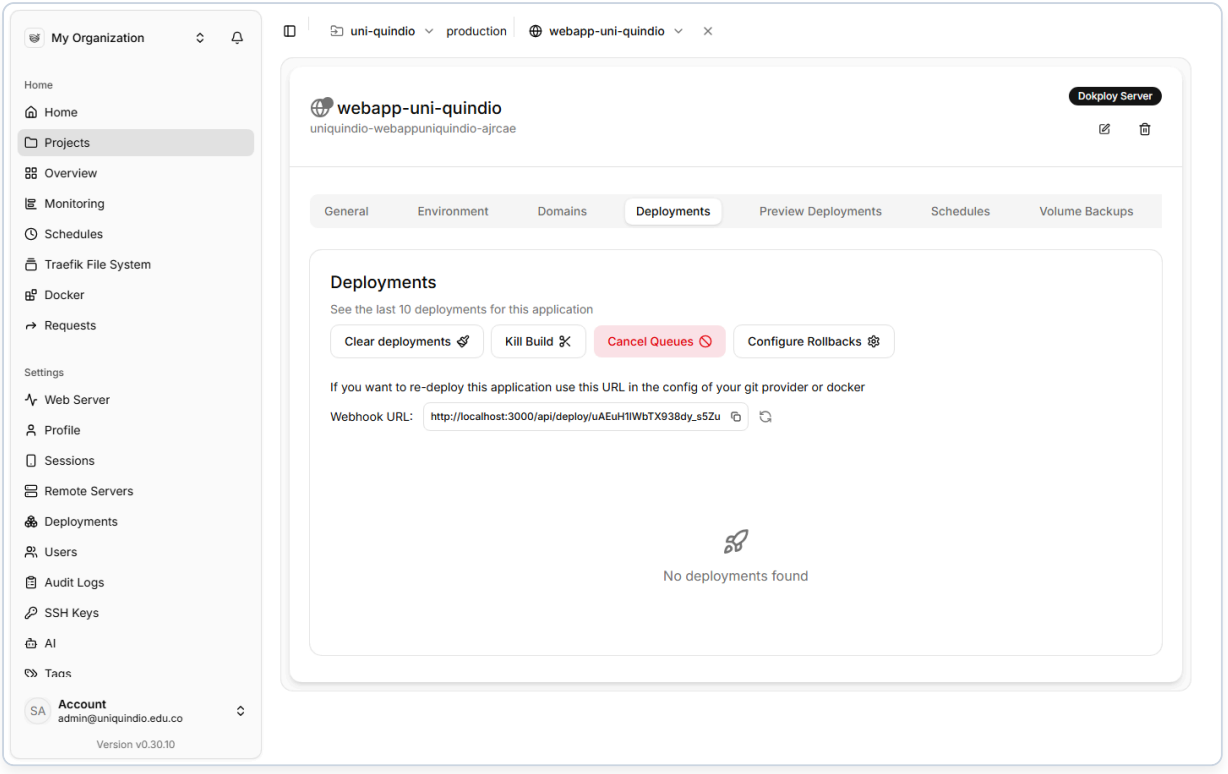


Figura: Consola de despliegue en tiempo real ejecutando Nixpacks y Swarm task rollout.

4.5 Verificación Funcional de Endpoints

La aplicación web fue verificada en el navegador web y mediante clientes HTTP, respondiendo satisfactoriamente en sus tres rutas:



Figura: Endpoint raíz (/) respondiendo con el mensaje institucional y estado del despliegue.

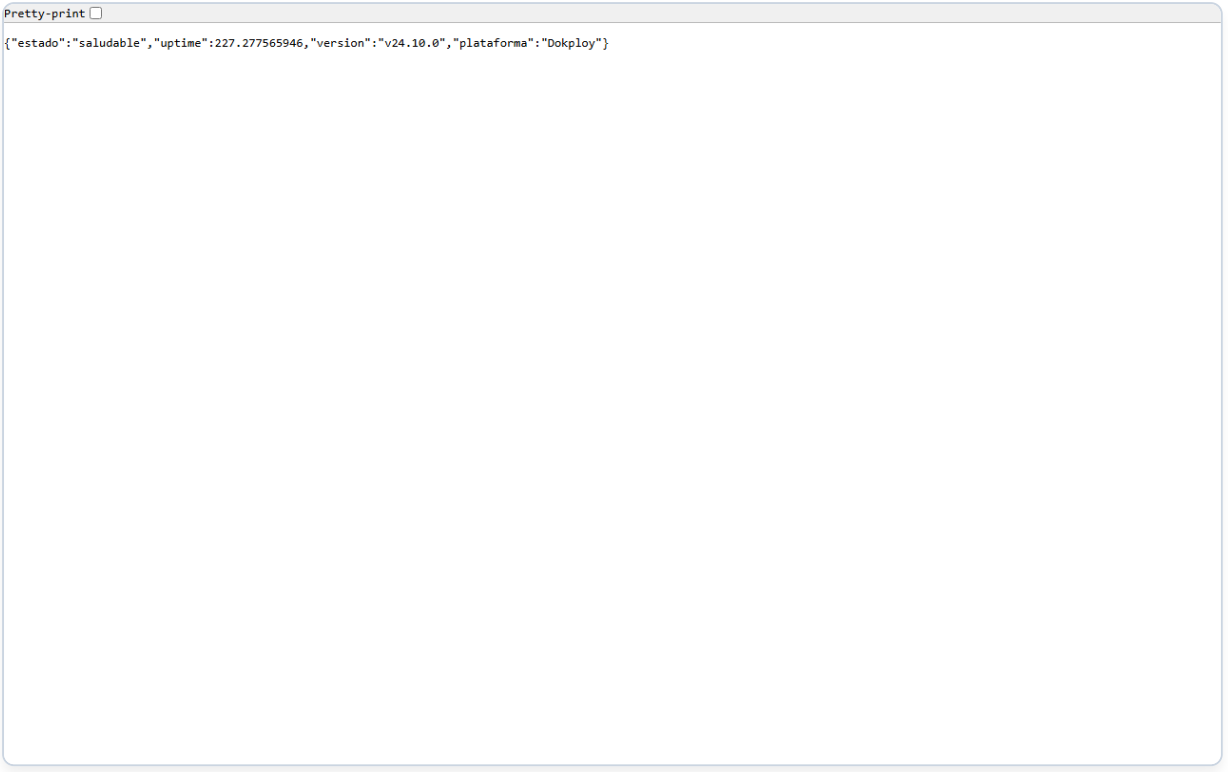


Figura: Endpoint /health indicando estado "saludable", uptime y versión de la plataforma.

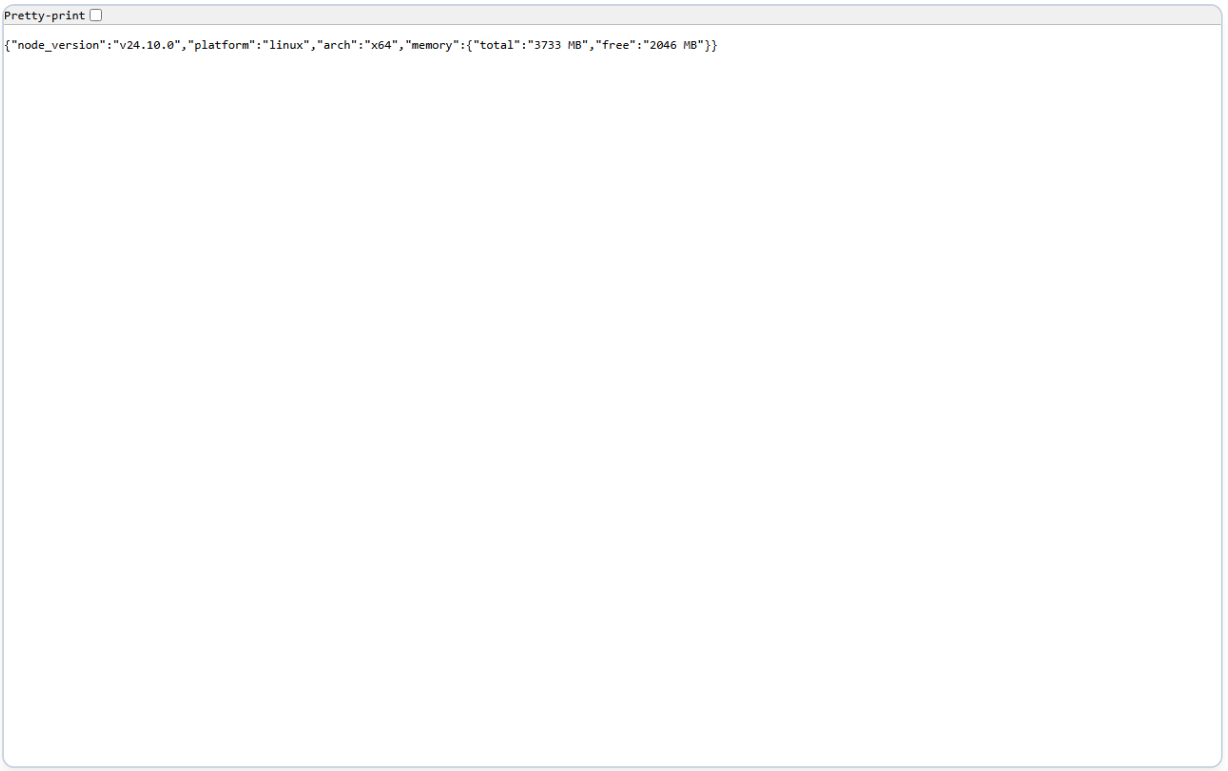


Figura: Endpoint /info mostrando telemetría del runtime de Node.js y memoria del host.

5. Fase 4: Despliegue de Bases de Datos Gestionadas

Dokploy permite aprovisionar bases de datos en contenedores con un solo clic, incluyendo volumen persistente administrado, generación de credenciales seguras, redes privadas y soporte para copias de seguridad.

5.1 Instancias Aprovisionadas

Dentro del proyecto `uni-quindio` se desplegaron tres motores de persistencia:

1. **MySQL 8.0 (`db-uni-quindio`):** Base de datos relacional para transacciones primarias. Base de datos: `uni_quindio_db`, Usuario: `admin_quindio`, Puerto: `3306`.
2. **PostgreSQL 16 (`db-postgres-quindio`):** Base de datos para analítica avanzada. Base de datos: `analytics_db`, Usuario: `analyst_quindio`, Puerto: `5432`.
3. **Redis 7 (`redis-cache-quindio`):** Almacén de datos en memoria para sesiones y caché distribuida. Puerto: `6379`.

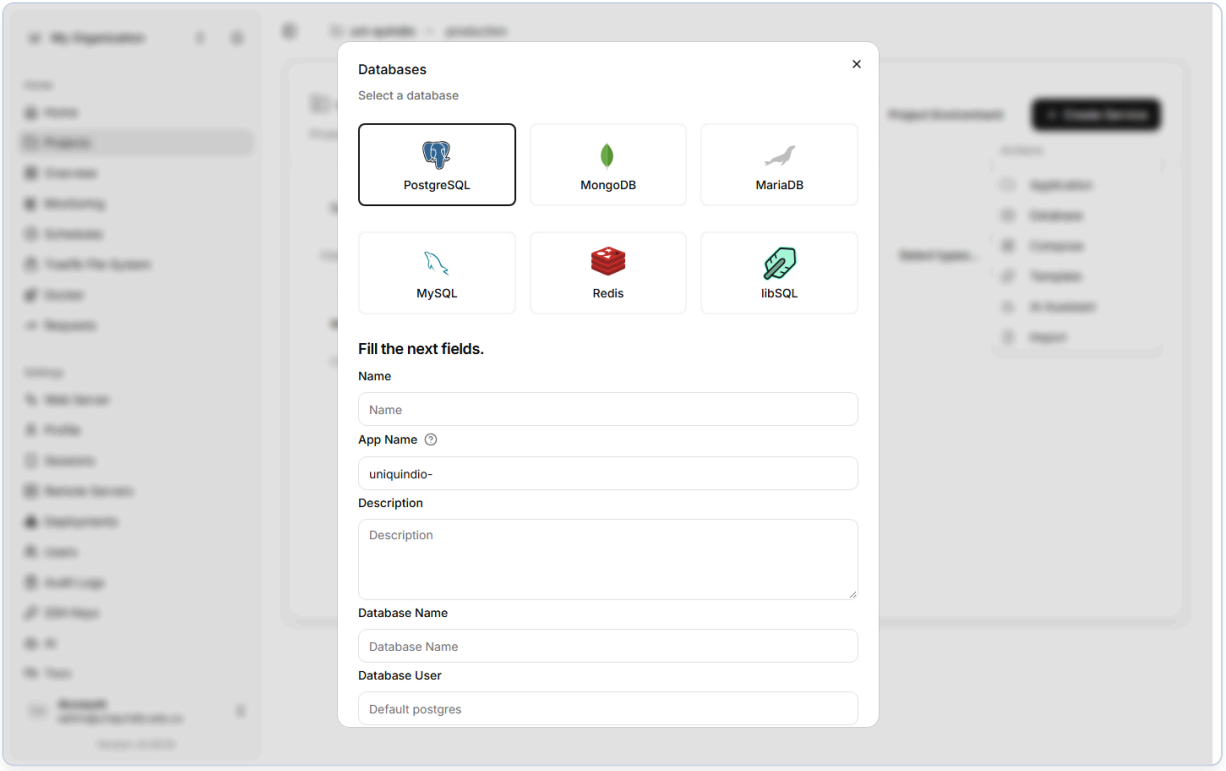


Figura: Modal de selección de base de datos gestionada en Dokploy.

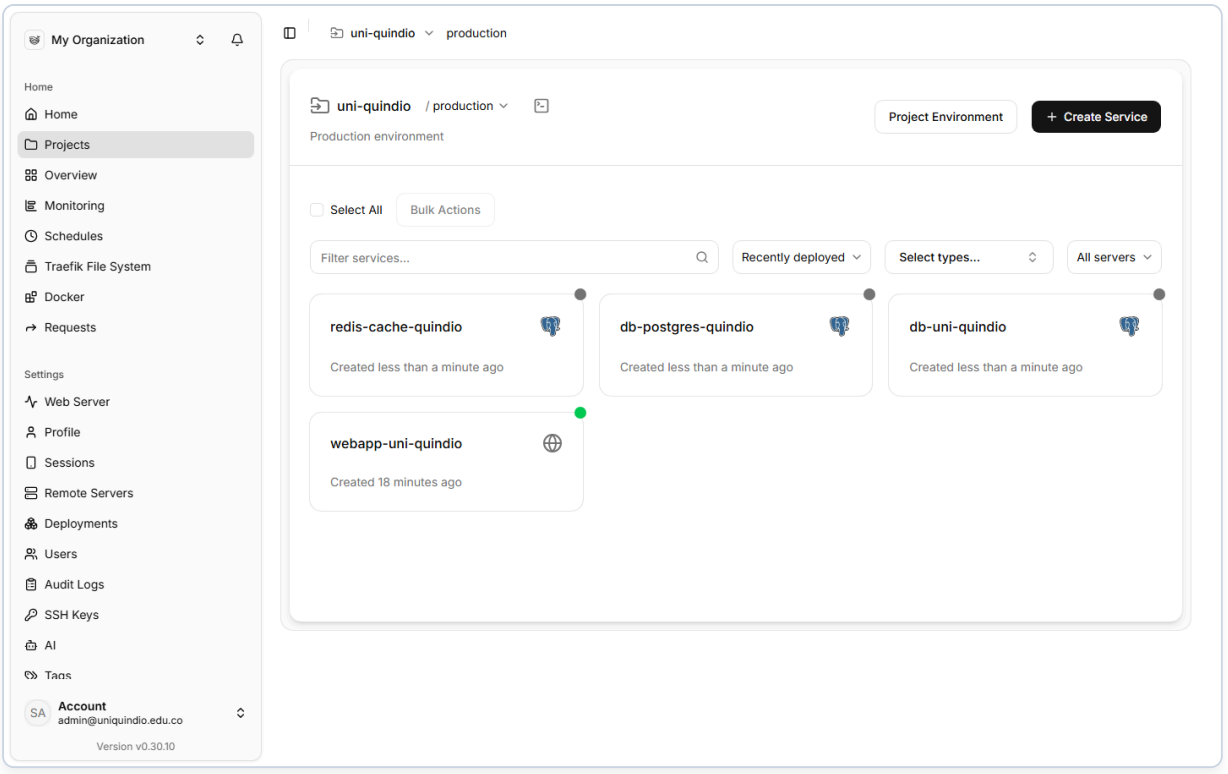


Figura: Panel del proyecto uni-quindio con los tres motores de bases de datos activos.

5.2 Detalles y Conectividad de los Servicios

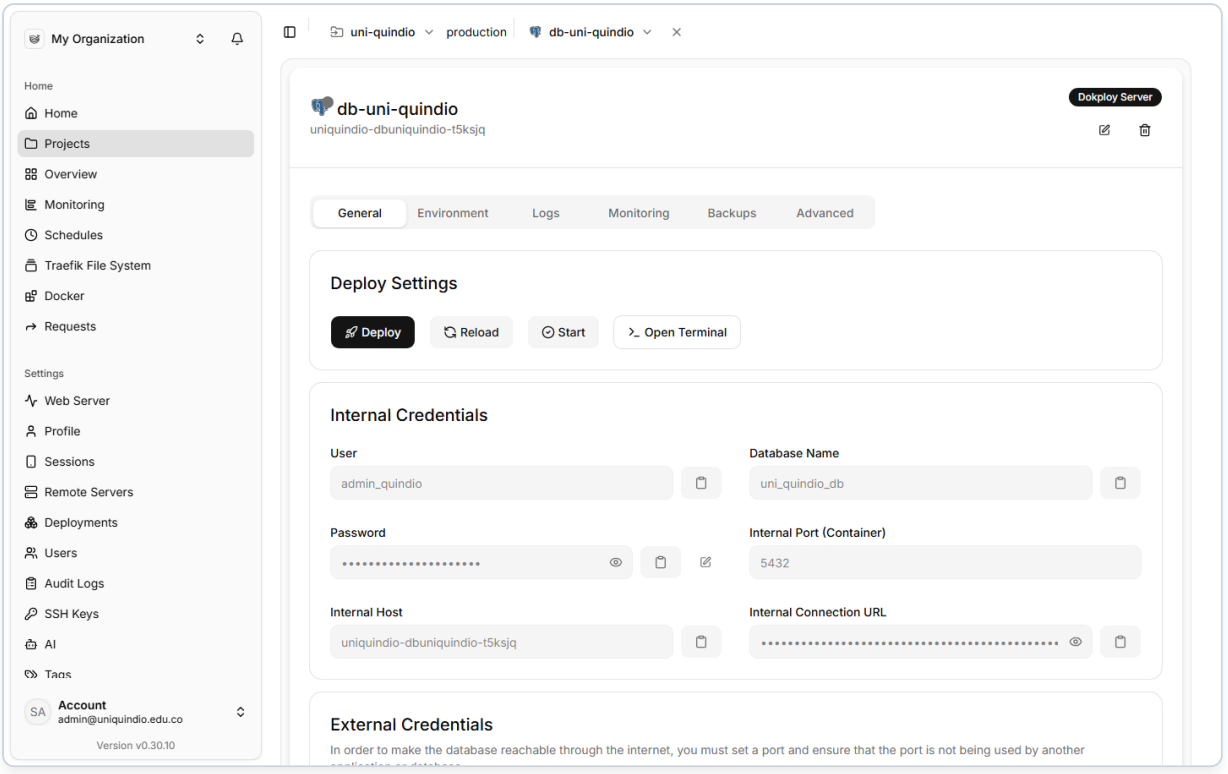


Figura: Detalle y parámetros de conexión de MySQL (db-uni-quindio).

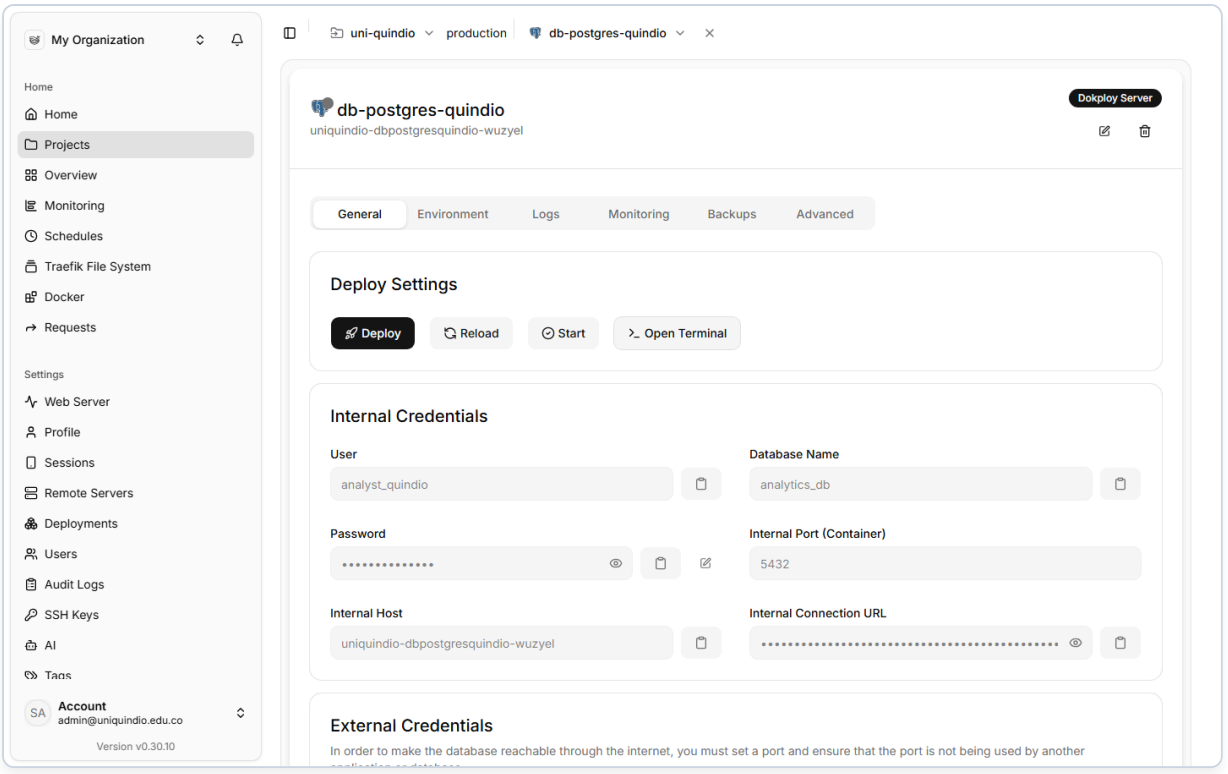


Figura: Detalle y parámetros de conexión de PostgreSQL (db-postgres-quindio).

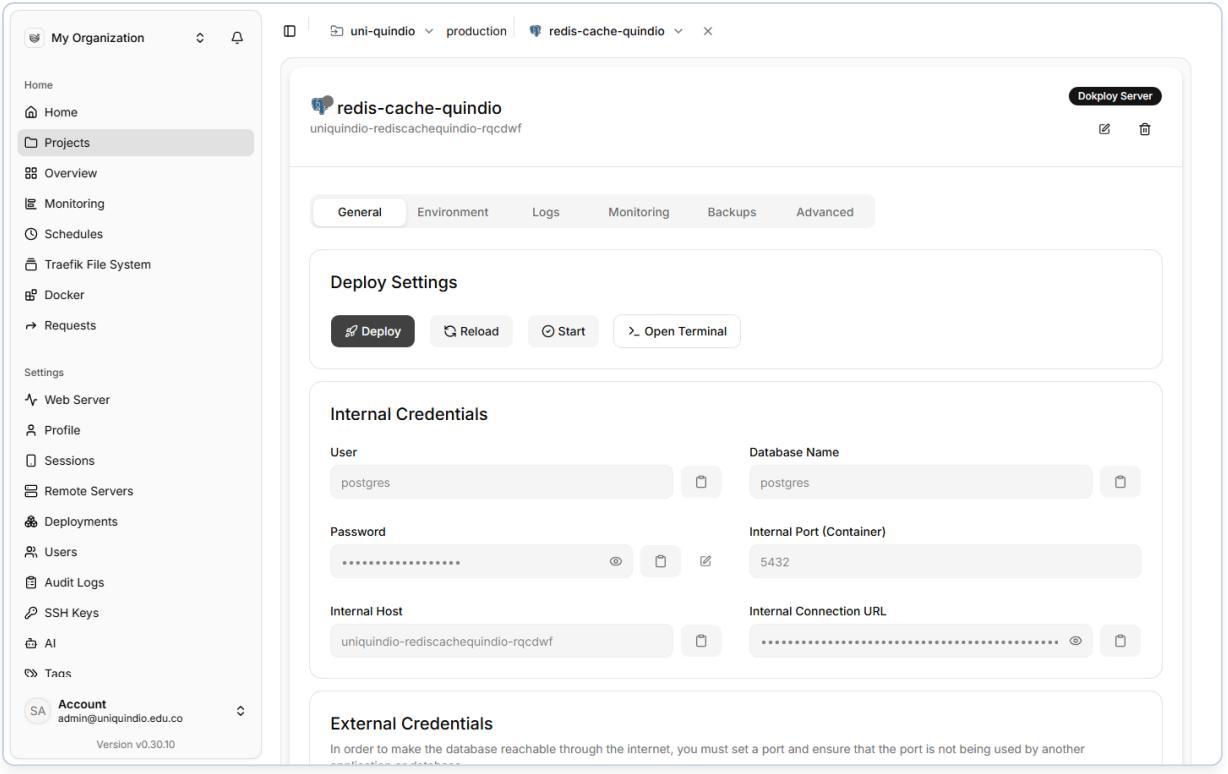


Figura: Detalle y parámetros de conexión de Redis (redis-cache-quindio).

5.3 Vinculación con la Aplicación Web

Se actualizaron las variables de entorno de `webapp-uni-quindio` para conectarse al servicio MySQL utilizando el nombre de host interno provisto por Docker Swarm (`uniquindio-dbuniquindio-t5ksjq`) a

través del puerto 3306, asegurando que el tráfico de la base de datos nunca viaje desprotegido a través de la red pública.

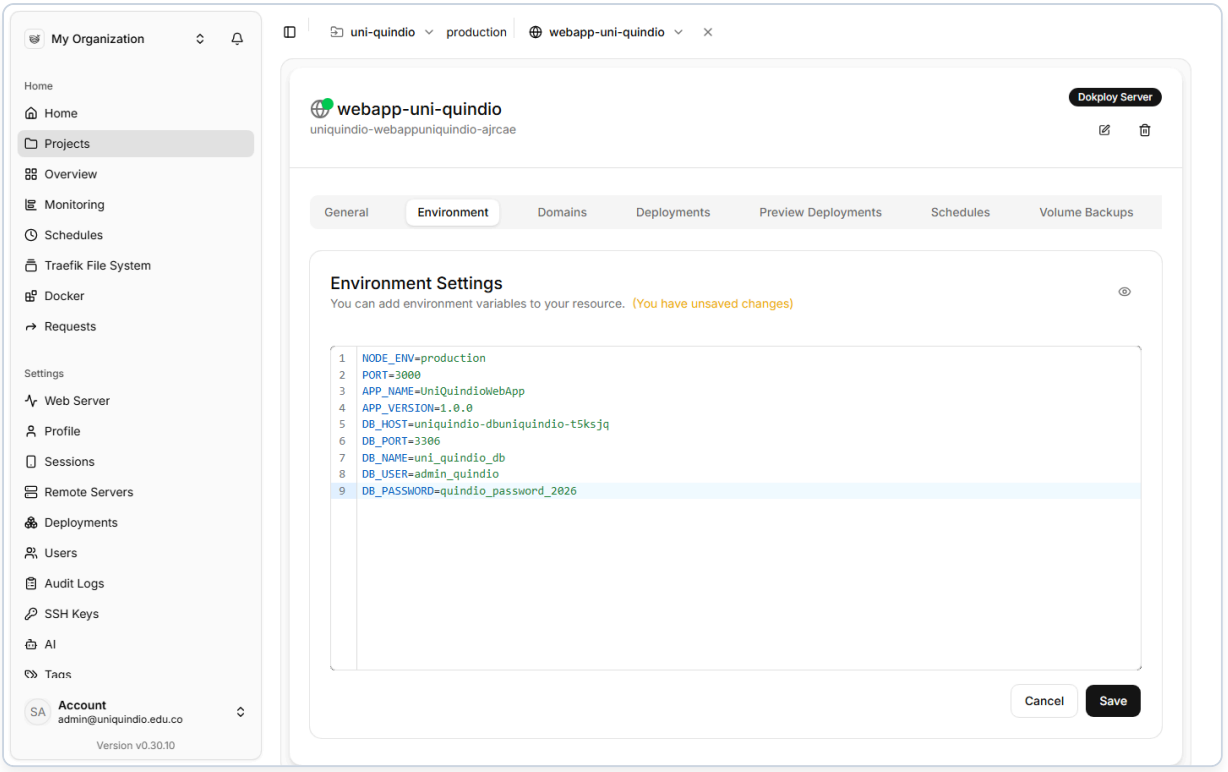


Figura: Variables de entorno actualizadas con los parámetros de conexión de la base de datos.

6. Fase 5: Despliegue con Docker Compose (Multi-Servicio)

En arquitecturas de microservicios modernas, las aplicaciones requieren el despliegue coordinado de múltiples capas interdependientes (frontend, backend, almacenamiento, interfaces de administración). Dokploy soporta de manera nativa la definición y despliegue de stacks completos mediante Docker Compose.

6.1 Arquitectura del Stack Multi-Servicio

Se creó un nuevo proyecto denominado `multi-service` y se definió un stack Compose compuesto por cuatro contenedores:

```
version: '3.8'

services:
  # Microservicio de aplicación web
  app:
    image: node:20-alpine
    container_name: compose-webapp
    ports:
      - "3001:3000"
    environment:
      - NODE_ENV=production
      - PORT=3000
    command: ["node", "-e", "require('http').createServer((req, res) ⇒ { res.writeHead(200, {'Content-T
    restart: unless-stopped

  # Base de datos NoSQL MongoDB 7
  mongodb:
    image: mongo:7
    container_name: compose-mongo
    environment:
      - MONGO_INITDB_ROOT_USERNAME=admin_compose
      - MONGO_INITDB_ROOT_PASSWORD=mongo_compose_2026
      - MONGO_INITDB_DATABASE=compose_db
    volumes:
      - mongo-data:/data/db
    restart: unless-stopped

  # Interfaz gráfica de administración phpMyAdmin
  phpmyadmin:
    image: phpmyadmin:latest
    container_name: compose-phpmyadmin
    ports:
      - "8080:80"
    environment:
      - PMA_HOST=db-service
      - PMA_PORT=3306
    depends_on:
      - db-service
    restart: unless-stopped

  # Base de datos relacional MySQL 8.0
  db-service:
    image: mysql:8.0
    container_name: compose-mysql
    environment:
      - MYSQL_ROOT_PASSWORD=root_compose_2026
      - MYSQL_DATABASE=compose_db
      - MYSQL_USER=admin_compose
      - MYSQL_PASSWORD=compose_pass_2026
    volumes:
      - mysql-data:/var/lib/mysql
    restart: unless-stopped

volumes:
  mongo-data:
  mysql-data:
```

6.2 Configuración y Despliegue en Dokploy

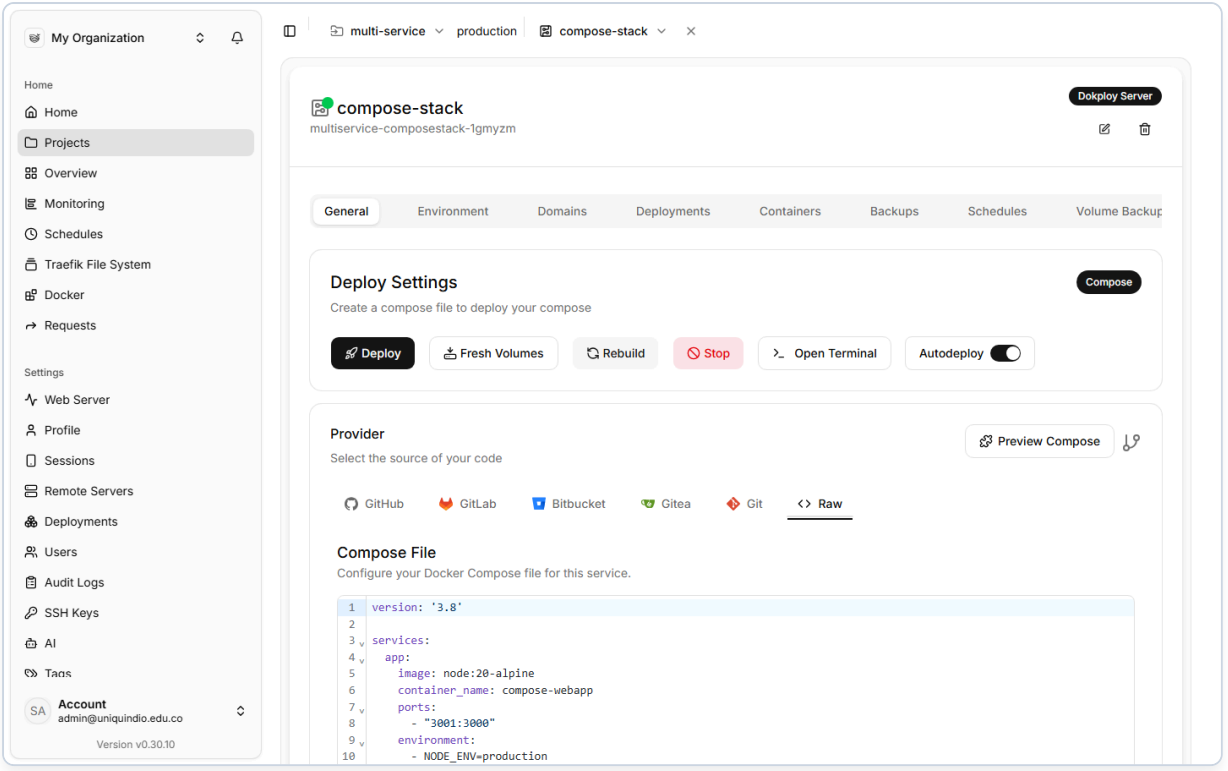


Figura: Editor YAML de Dokploy con la definición del stack Compose configurado.

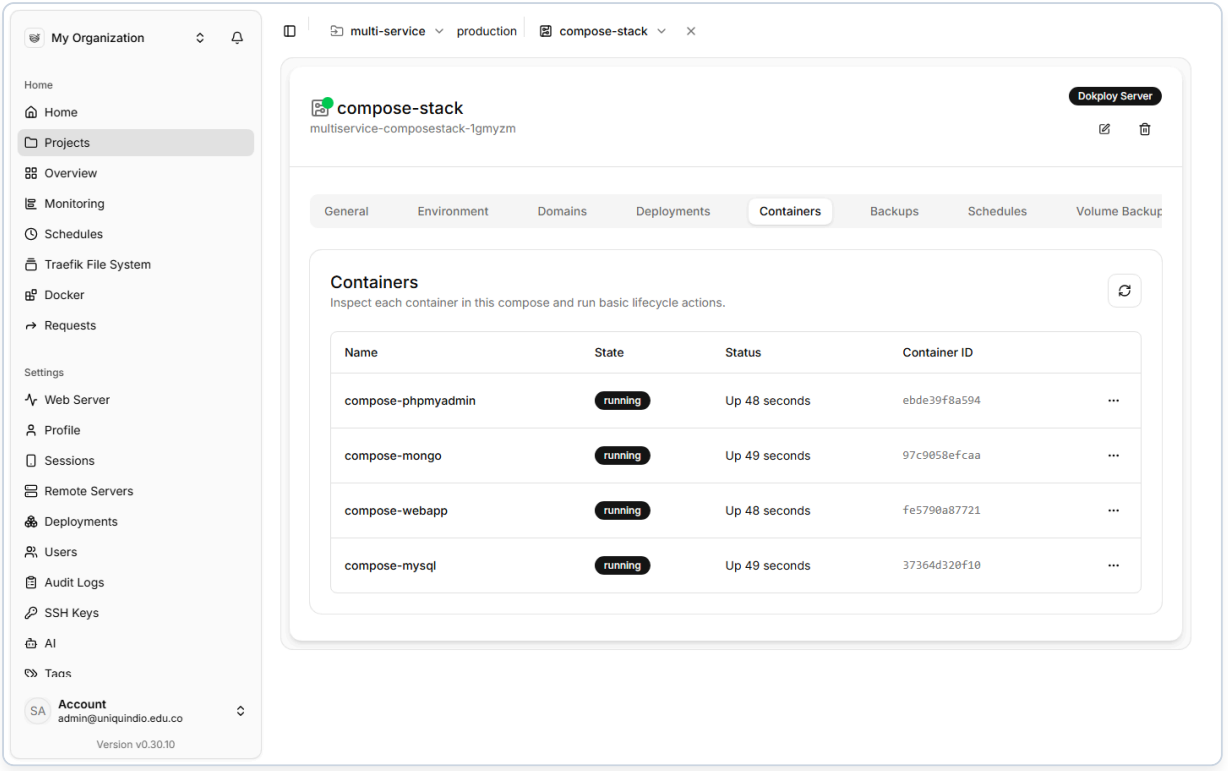


Figura: Pestaña Containers de Dokploy confirmando los 4 microservicios en ejecución.

6.3 Verificación de Acceso Web

Se comprobó el correcto funcionamiento de los servicios expuestos: la aplicación web en el puerto 3001 y la consola web de administración phpMyAdmin en el puerto 8080.

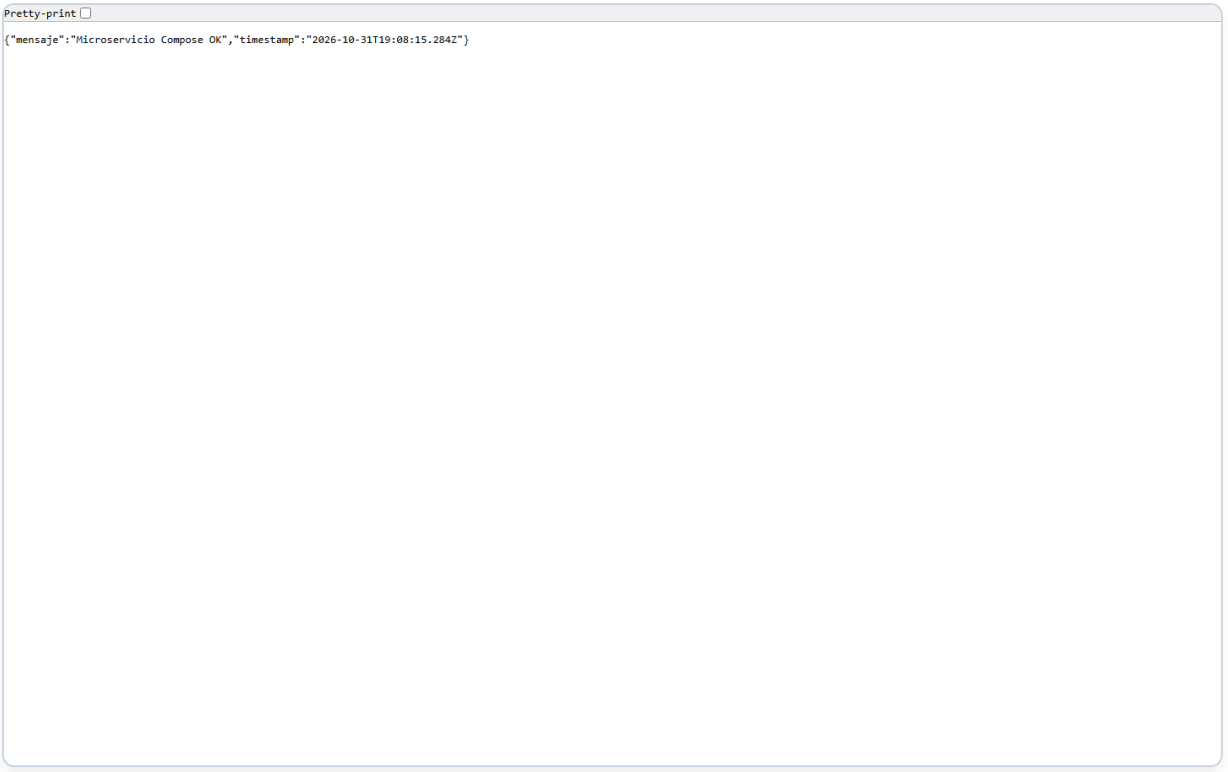


Figura: Acceso HTTP al microservicio Compose en http://localhost:3001/.

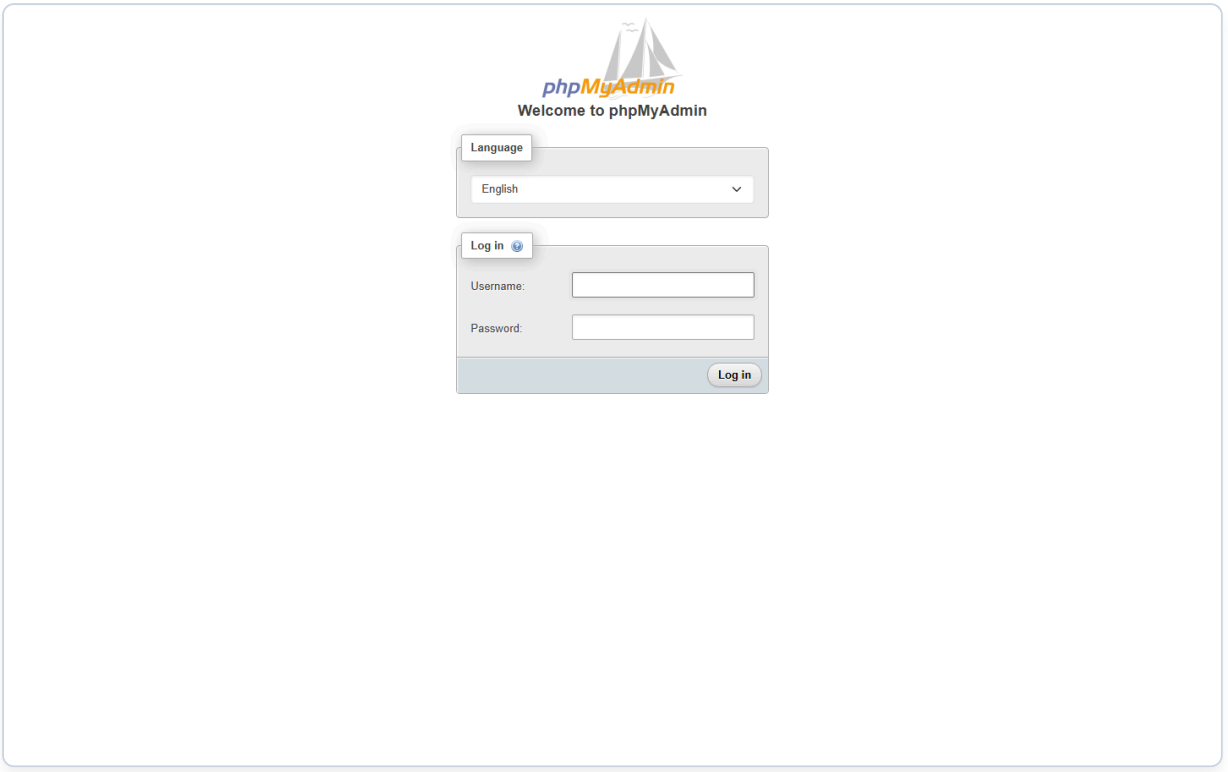


Figura: Interfaz de acceso a phpMyAdmin en http://localhost:8080/ vinculada a MySQL.

7. Fase 6: Monitoreo, Logs y Backups

La observabilidad y la continuidad de negocio son pilares indispensables en la administración de infraestructura de TI. Dokploy integra herramientas nativas para la supervisión de telemetría, inspección de registros en vivo y automatización de respaldos.

7.1 Monitoreo de Recursos y Métricas

El módulo de monitoreo captura métricas en tiempo real tanto a nivel global del servidor como de manera granular por cada contenedor: porcentaje de uso de CPU, consumo de memoria RAM, operaciones de entrada/salida en disco (I/O) y tráfico de red entrante y saliente.

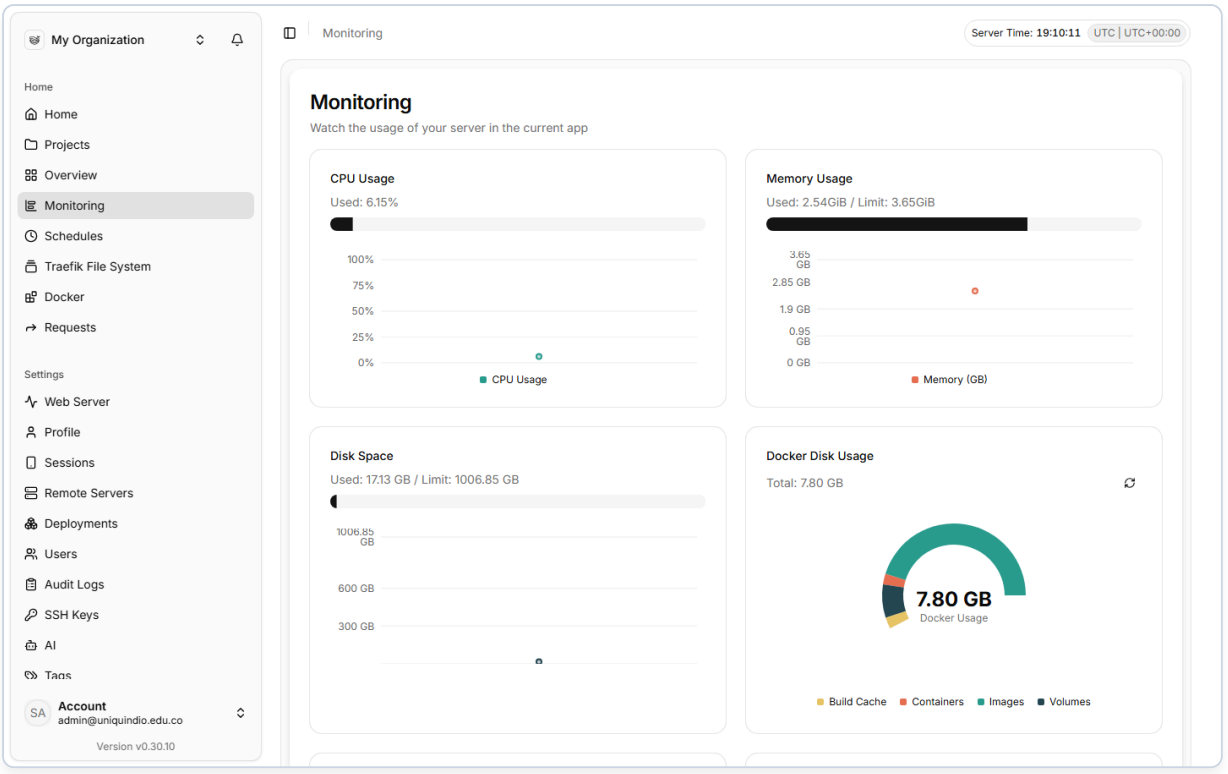


Figura: Dashboard de monitoreo general del servidor con gráficas de CPU, RAM, Disco y Red.

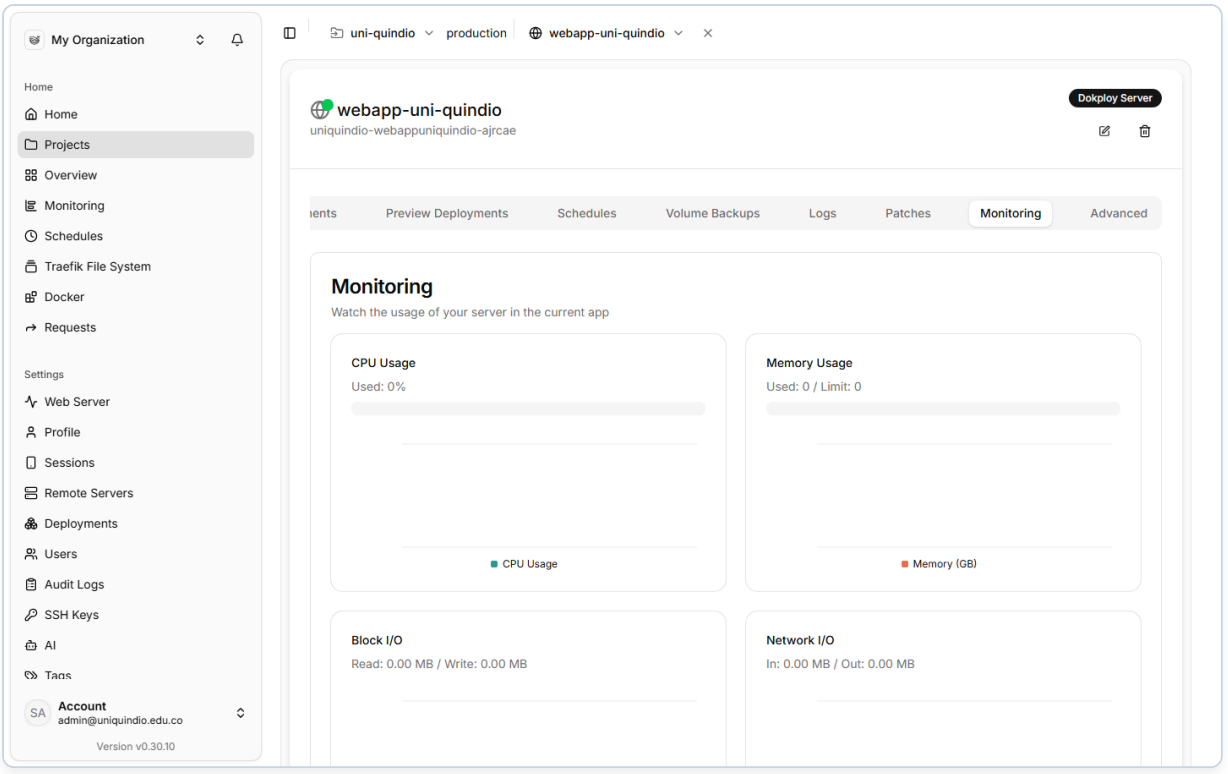


Figura: Monitoreo granular del contenedor de la aplicación web en Dokploy.

7.2 Visualización Centralizada de Logs

Dokploy implementa un visor de registros en tiempo real conectado al stream de Docker/Swarm, lo que permite a los operadores diagnosticar errores de arranque, excepciones no capturadas o peticiones HTTP entrantes sin necesidad de abrir una sesión SSH en el servidor.

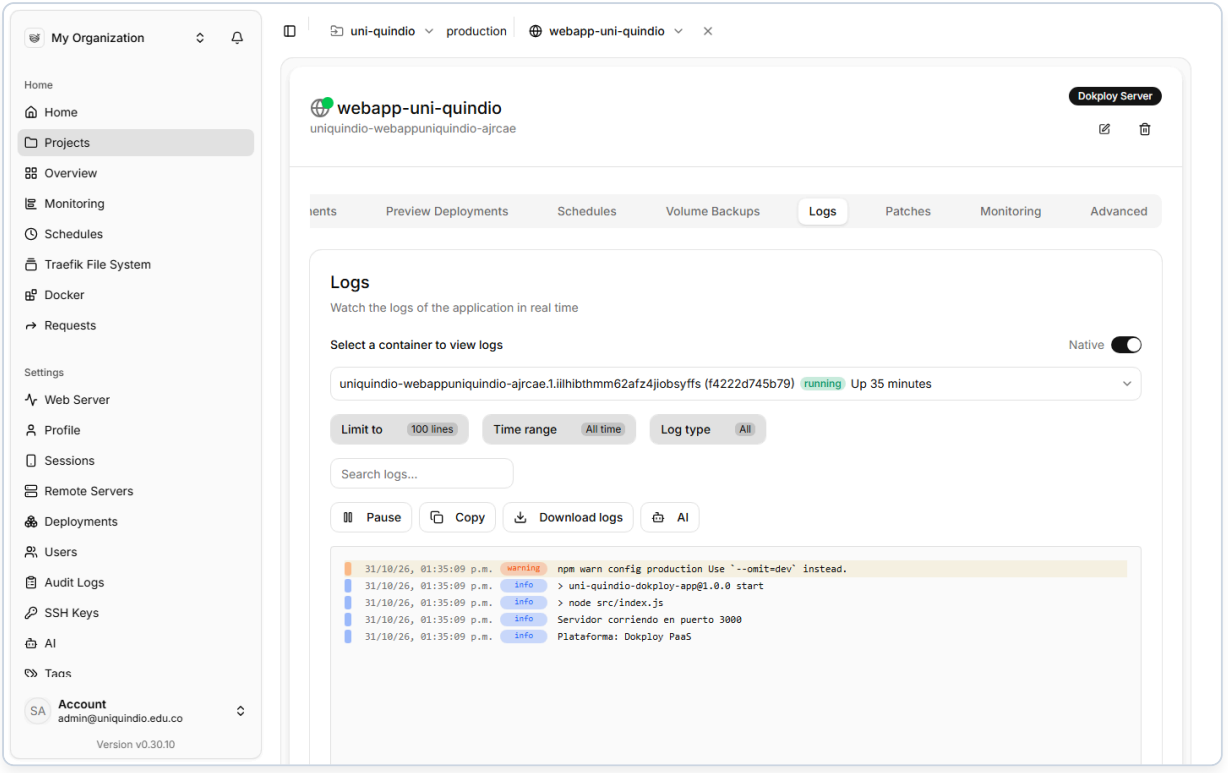


Figura: Visualizador de logs en tiempo real de la aplicación web en Dokploy.

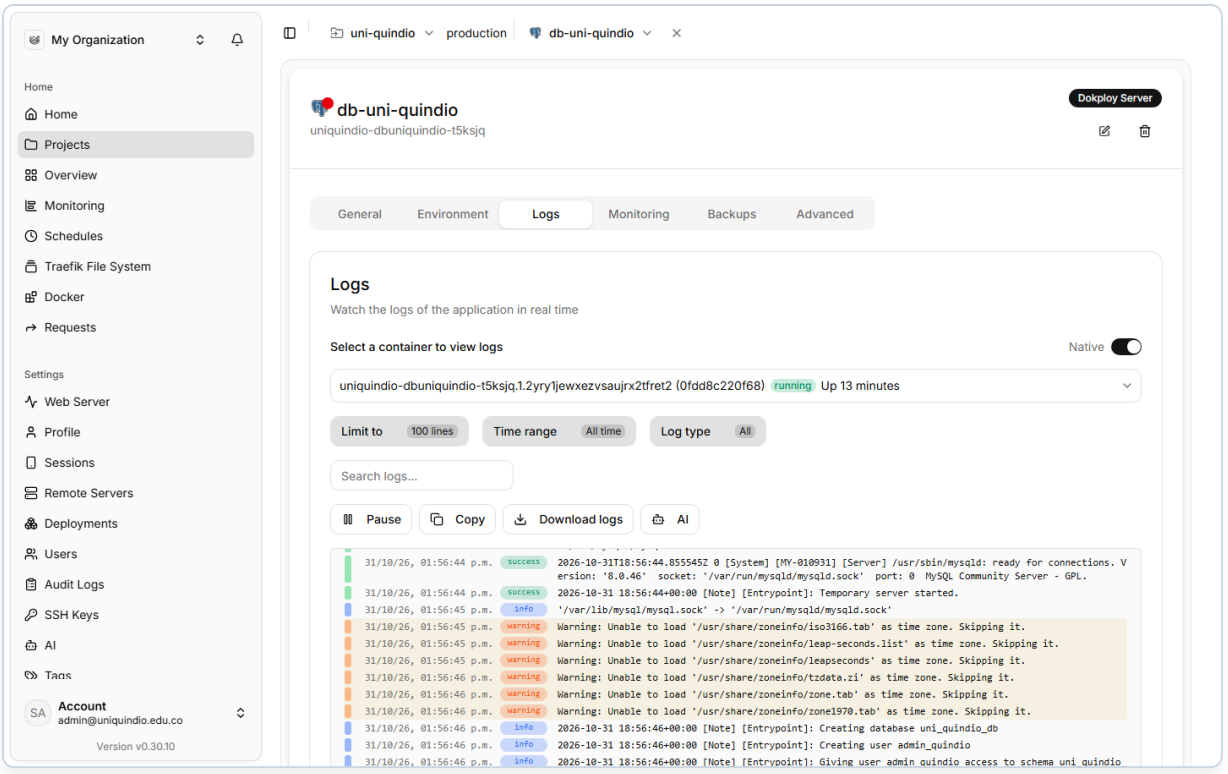


Figura: Inspección de logs de inicialización y conexión del motor MySQL.

7.3 Configuración de Copias de Seguridad (Backups)

El módulo de respaldos de Dokploy permite definir copias automatizadas de bases de datos mediante `cron` y exportarlas a destinos de almacenamiento de objetos compatibles con S3 (AWS S3, MinIO, Cloudflare R2, Backblaze B2).

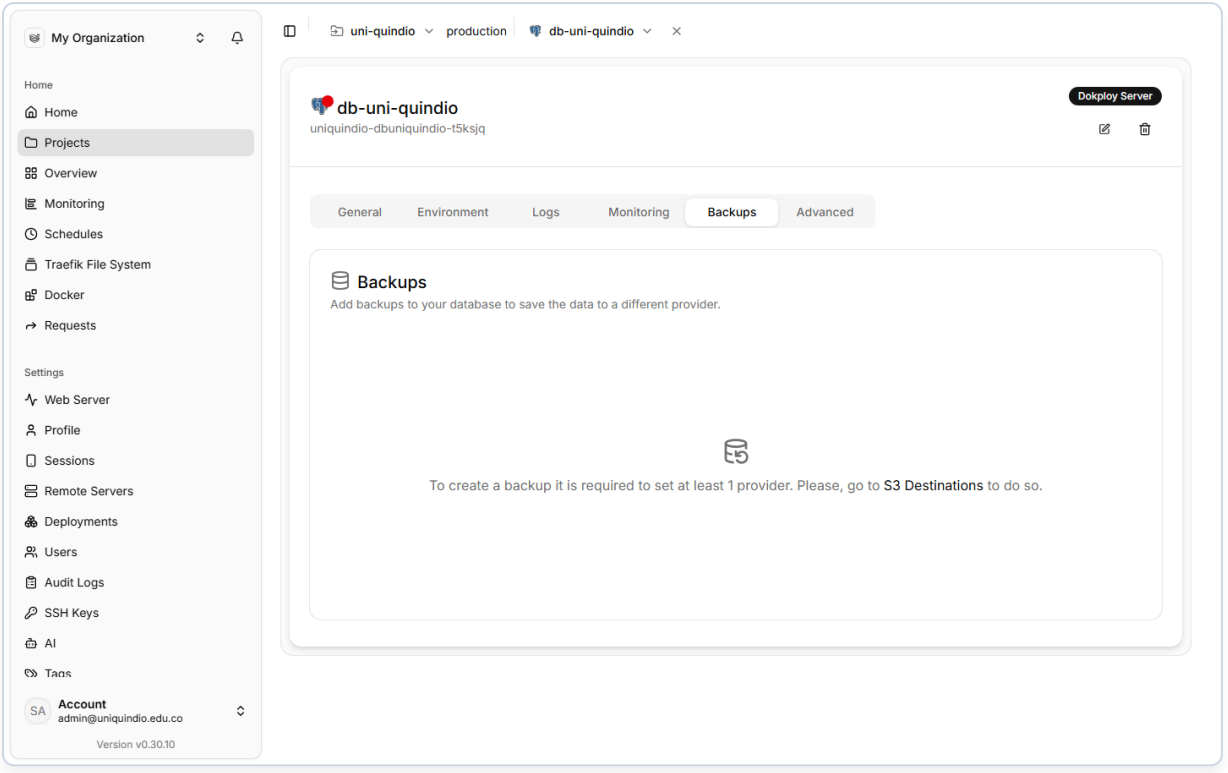


Figura: Pestaña de gestión de backups de base de datos en Dokploy.

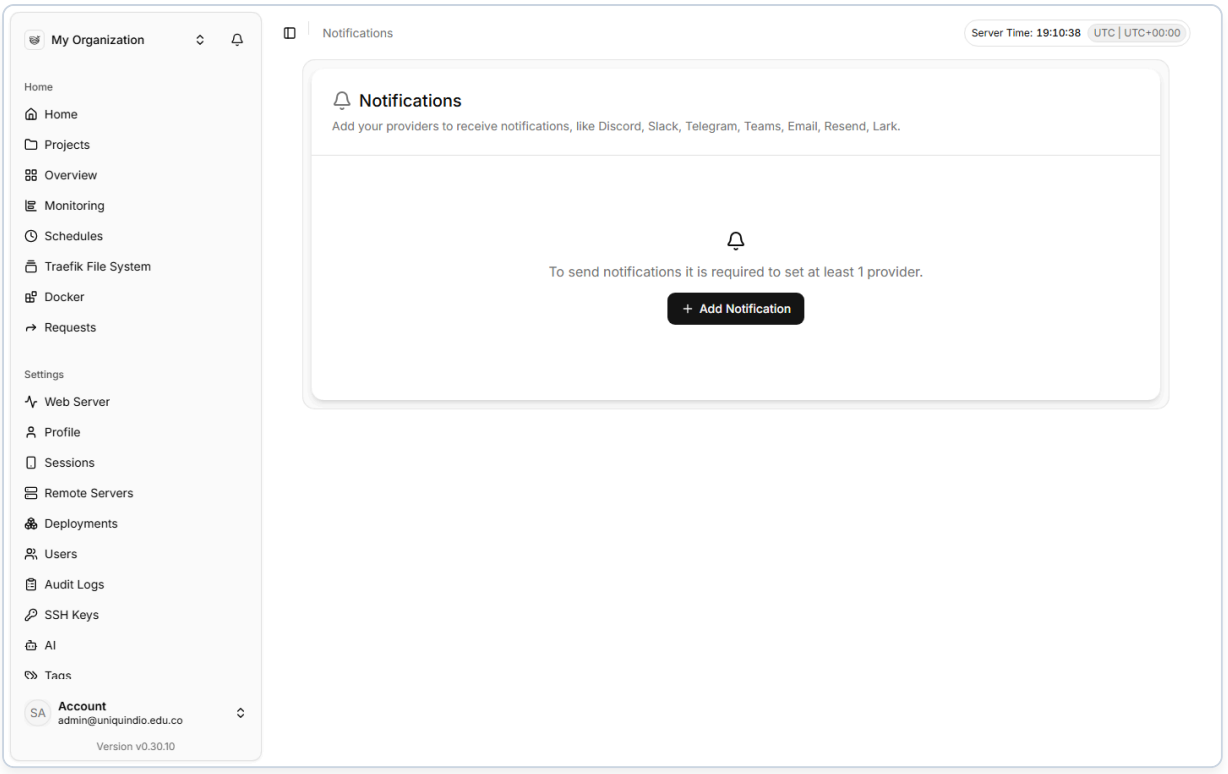


Figura: Configuración de canales de notificación (Discord, Telegram, Slack, Email) para alertas de backups y fallos.

8. Fase 7: Integración Git y Flujo de Auto-Despliegue (CI/CD)

Para eliminar la fricción operativa del despliegue manual, Dokploy incorpora capacidades de Integración y Despliegue Continuo (CI/CD). Cada aplicación dispone de una URL de webhook segura y un mecanismo de detección de cambios en ramas de Git.

8.1 Configuración del Webhook y Autodeploy

En la pestaña de *Deployments* de `webapp-uni-quindio`, se activó la opción **Autodeploy** y se obtuvo el endpoint de webhook generado criptográficamente.

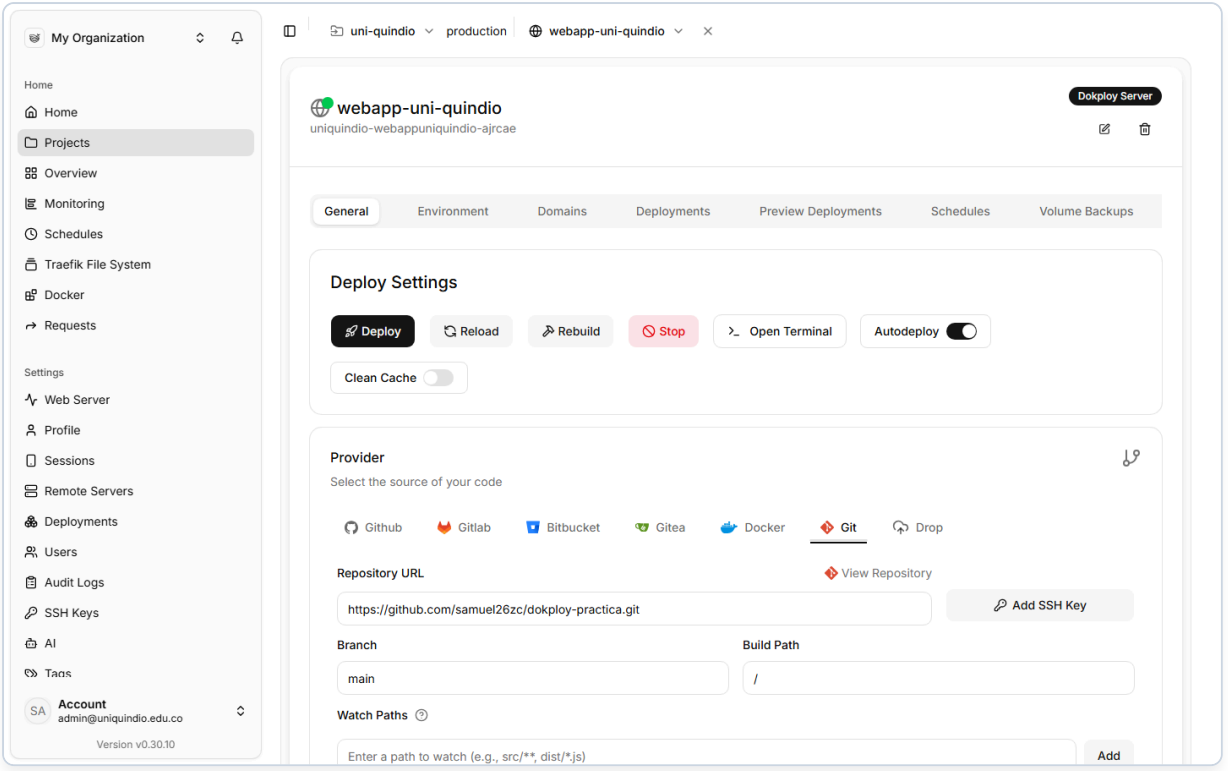


Figura: Configuración del switch Autodeploy y selección de rama Git.

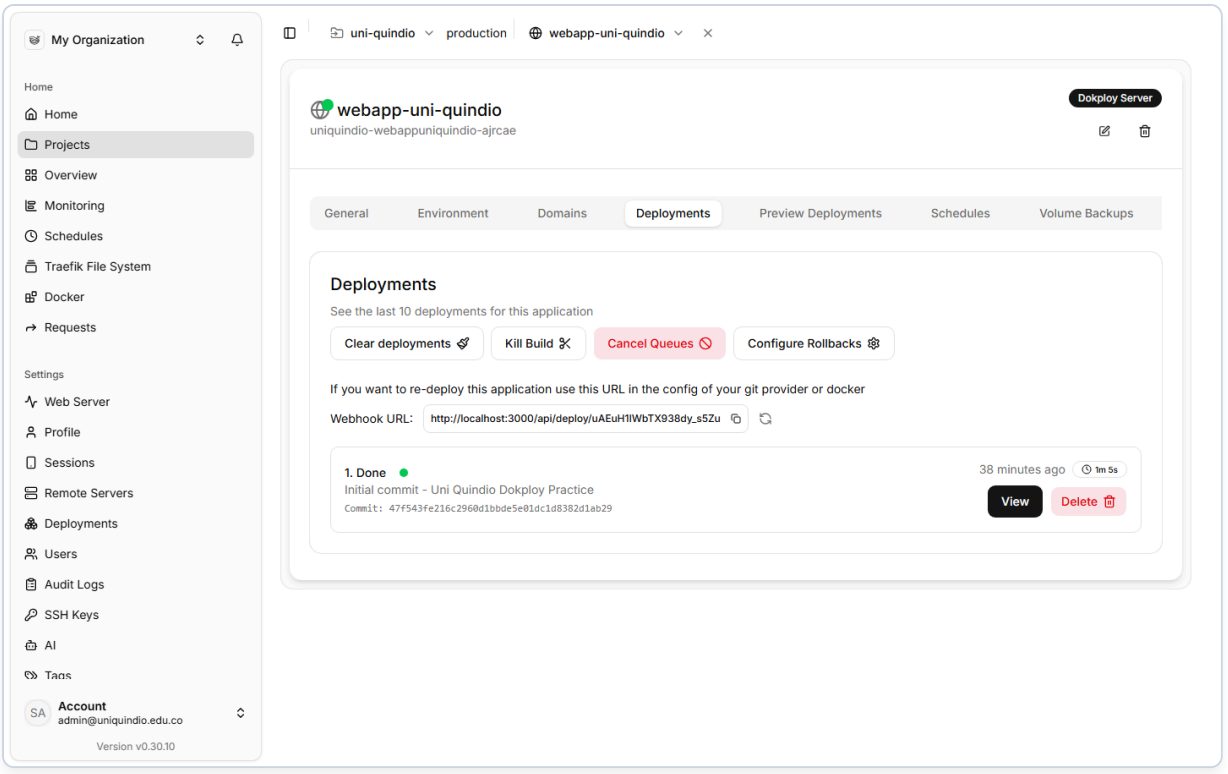


Figura: URL del webhook de despliegue automatizado provista por Dokploy.

8.2 Prueba del Ciclo CI/CD: Actualización a v2.0.0

Para validar el pipeline, se realizaron cambios en el código local de la aplicación: se incrementó la versión a `2.0.0` en `package.json` y se agregaron campos al payload JSON en `src/index.js`. Posteriormente, los cambios fueron enviados a GitHub:

```
git add .
git commit -m "feat: actualizar aplicación a v2.0.0 (Fase 7 CI/CD)"
git push origin main
```

Al dispararse la notificación hacia el webhook de Dokploy, la plataforma ejecutó automáticamente el pipeline: clonado del commit más reciente, compilación con Nixpacks, prueba de healthcheck y reemplazo en caliente del contenedor en Docker Swarm mediante Rolling Update, garantizando cero tiempo de inactividad (Zero Downtime).

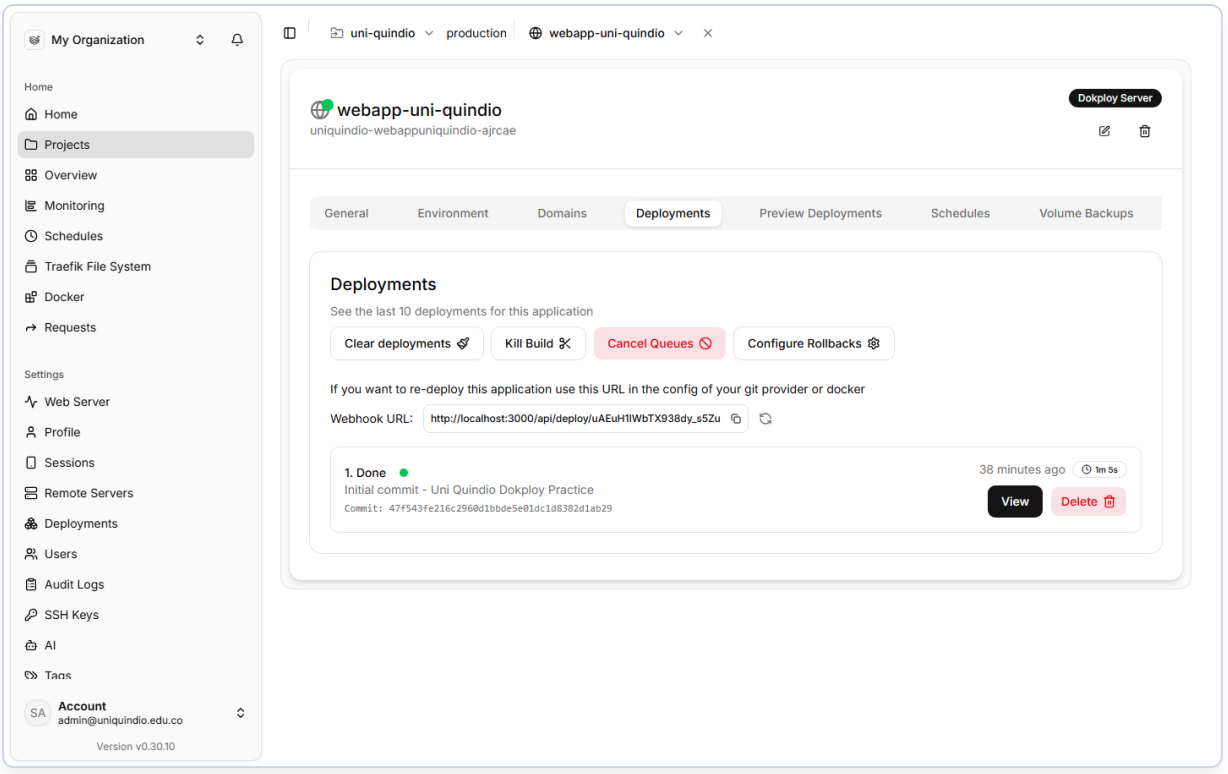


Figura: Historial de despliegues en Dokploy evidenciando las versiones v1.0.0 y v2.0.0 completadas exitosamente.

8.3 Verificación de la Versión 2.0.0 en Producción

Al refrescar la aplicación en el dominio público simulado `http://webapp.127.0.0.1.traefik.me/`, el servicio entregó de forma inmediata la nueva versión v2.0.0:

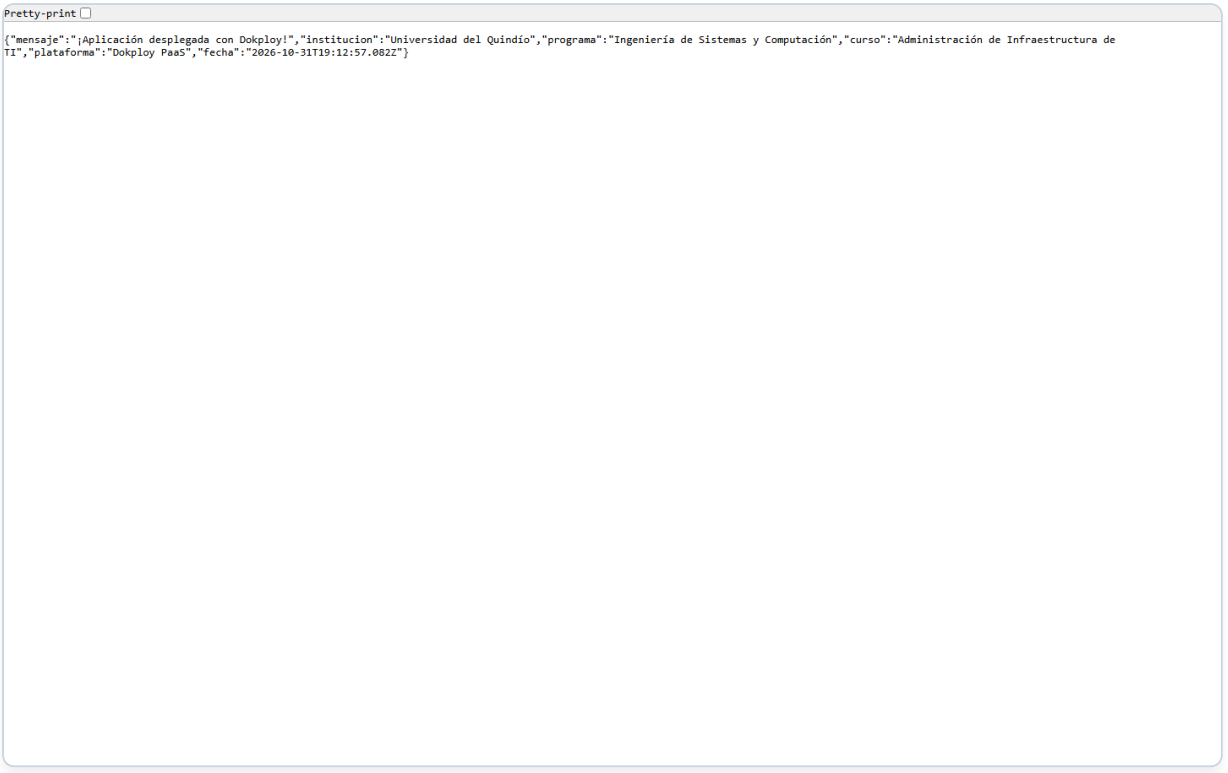


Figura: Endpoint raíz reflejando la versión 2.0.0 y confirmación de auto-despliegue CI/CD.

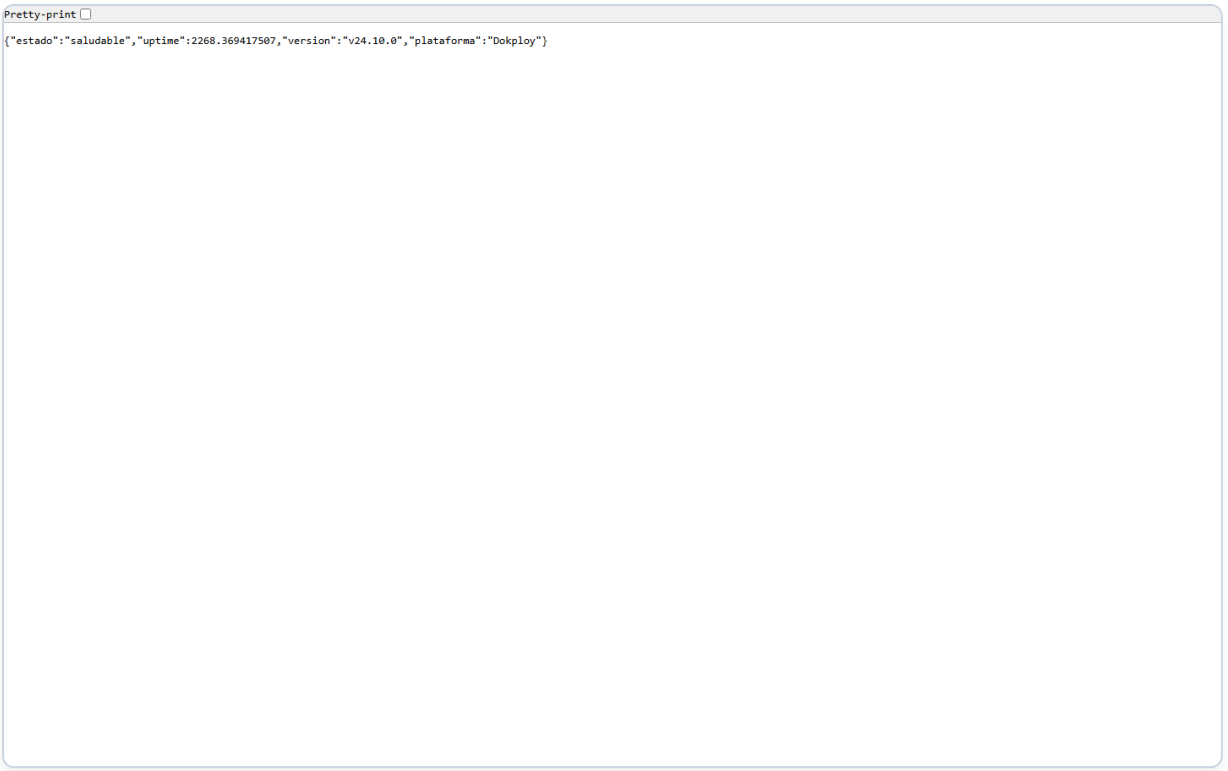


Figura: Endpoint /health confirmando app_version: 2.0.0 en estado saludable.

9. Registro de Problemas Encontrados y Soluciones Técnicas Aplicadas

Durante el desarrollo integral de la práctica se identificaron y solucionaron diversos desafíos técnicos de arquitectura, concurrencia y orquestación:

Problema Identificado	Causa Raíz	Solución Técnica Implementada
Fallo en tareas Swarm: *"No such container"*	Incompatibilidad de Docker 29 con el plugin `containerd-snapshotter` habilitado por defecto al interactuar con servicios Swarm.	Se editó `/etc/docker/daemon.json` fijando `"containerd-snapshotter": false` y se reinició el servicio Docker, restableciendo el driver estándar de almacenamiento.
Cierre de sesión prematuro en WSL 2	El subsistema WSL 2 suspende distros inactivas cuando no hay procesos interactivos activos en una terminal.	Se creó un proceso persistente en segundo plano (daemon) mediante `exec sleep infinity` para garantizar la permanencia activa de los contenedores y servicios.
Fallo en contenedor MySQL (Exit 1)	El contenedor oficial de MySQL 8.0 requiere estrictamente una contraseña de superusuario (`MYSQL_ROOT_PASSWORD`) durante el primer inicio para inicializar el diccionario de datos.	Se inyectó la variable `MYSQL_ROOT_PASSWORD` en el servicio gestionado de Dokploy, permitiendo al contenedor completar el script de inicialización sin abortar.
Timeout de navegación en pruebas automatizadas (Playwright)	Dokploy mantiene conexiones continuas mediante WebSockets y Server-Sent Events (SSE) para logs y telemetría, impidiendo que la condición `networkidle` se complete.	Se sustituyó la condición de espera por `waitUntil: 'domcontentloaded'` combinada con selectores reactivos explícitos, garantizando la captura estable de pantallas.
Error de sintaxis YAML en Docker Compose	El uso de dos puntos (`:`) y comillas anidadas en un comando en línea `command: sh -c "echo 'const http = ...' ..."` fue interpretado erróneamente por el parser de Dokploy como una clave anidada.	Se reformuló el comando utilizando sintaxis de arreglo estándar de YAML: `command: ["node", "-e", "..."]`, permitiendo un parseo perfecto y el arranque de todos los microservicios.
Autenticación en GitHub API sin cliente gh	El binario `gh` no se encontraba preinstalado en el PATH del entorno Windows para la creación automatizada del repositorio.	Se extrajo el token OAuth de GitHub almacenado de manera segura en el Administrador de Credenciales de Windows y se interactuó directamente contra la REST API v3 de GitHub mediante PowerShell.

10. Respuestas al Cuestionario de Análisis

A continuación se presentan las respuestas fundamentadas, técnicas y exhaustivas a las ocho preguntas de análisis formuladas en la guía de laboratorio:

1. ¿Qué ventajas ofrece Dokploy frente al despliegue manual con Docker CLI?

El despliegue manual con Docker CLI requiere que el administrador gestione de forma artesanal la creación de redes virtuales, configuración de mapeo de puertos estáticos, generación y renovación periódica de certificados SSL con Certbot, configuración manual de proxies inversos (Nginx/Traefik) y ejecución manual de compilaciones de imágenes. Además, carece de interfaz centralizada para observar telemetría y administrar variables de entorno seguras.

Dokploy abstrae toda esta complejidad operativa al proporcionar:

- **Aprovisionamiento automatizado de proxy:** Integra Traefik v3 de forma nativa, detectando nuevos contenedores y generando rutas dinámicas sin recargar servidores web.
- **Compilación agnóstica con Nixpacks:** Convierte código fuente de múltiples lenguajes (Node, Python, Go, PHP, etc.) en imágenes de producción óptimas sin requerir Dockerfiles manuales.
- **Gestión de ciclo de vida CI/CD:** Proporciona webhooks para auto-despliegue continuo con rolling updates (despliegue progresivo sin caída de servicio).
- **Consola unificada:** Permite gestionar secretos, proyectos, bases de datos con un clic, tareas cron y copias de seguridad desde un único panel con control de acceso basado en roles (RBAC).

2. ¿Cómo gestiona Dokploy el enrutamiento y los certificados SSL a través de Traefik?

Dokploy utiliza **Traefik v3** como su motor de borde (Edge Router) y reverse proxy dinámico. Traefik escucha constantemente el socket de Docker (`/var/run/docker.sock`) e inspecciona las etiquetas (Docker labels) o archivos de configuración dinámica generados por Dokploy en `/etc/dokploy/traefik` .

Cuando un usuario añade un dominio a una aplicación o servicio:

1. Dokploy genera dinámicamente un *Router* y un *Service* en la configuración de Traefik, especificando la regla de coincidencia (ej. `Host(`webapp.127.0.0.1.traefik.me`)`) y el puerto interno del contenedor.
2. Para entornos de producción con dominios públicos, Traefik se comunica directamente con las autoridades certificadoras mediante el protocolo **ACME (Let's Encrypt / ZeroSSL)** usando desafíos HTTP-01 o DNS-01.
3. Traefik almacena los certificados TLS de forma centralizada en un archivo JSON seguro (`acme.json`), gestiona automáticamente su renovación 30 días antes del vencimiento y aplica redirección forzada de HTTP (puerto 80) a HTTPS (puerto 443) con soporte para TLS 1.3 y HSTS.

3. ¿Qué diferencias existen entre desplegar una aplicación individual y un stack con Docker Compose en Dokploy?

Característica	Aplicación Individual en Dokploy	Stack Docker Compose en Dokploy
Modelo de Ejecución	Se despliega como un *Docker Swarm Service* con réplicas gestionadas y alta disponibilidad de tareas.	Se despliega utilizando el binario `docker compose` estándar como un conjunto de contenedores tradicionales.
Fuente de Construcción	Soporta Git con Nixpacks, Buildpacks, Dockerfile específico o imágenes pre-compiladas de Docker Hub/GHCR.	Basado en un archivo declarativo `docker-compose.yml` (Raw o sincronizado desde Git).
Topología de Red	Se conecta automáticamente a `dokploy-network` y crea una red overlay propia por servicio.	Permite definir múltiples redes personalizadas (bridge, overlay), enlaces inter-contenedores y directivas `depends_on`.
Escalabilidad	Escalado horizontal nativo modificando el número de réplicas en Swarm con balanceo de carga integrado.	El escalado depende de directivas de Compose o replicación estática de puertos.
Casos de Uso Ideales	APIs independientes, frontends web, microservicios stateless desacoplados.	Stacks complejos acoplados localmente (ej. WordPress + MySQL, PHP + Redis + Worker, sistemas multi-nivel).

4. ¿Cómo maneja Dokploy las variables de entorno y los secretos? ¿Es seguro para entornos de producción?

Dokploy almacena las variables de entorno en su base de datos relacional interna PostgreSQL. En la interfaz gráfica, los valores de los secretos son enmascarados visualmente mediante campos de tipo clave o editores protegidos con toggles de visibilidad.

Durante el proceso de despliegue, Dokploy genera un archivo `.env` aislado con permisos restrictivos en el directorio del servicio (`/etc/dokploy/...`) o inyecta las variables directamente en la definición de la tarea de Swarm/Docker en memoria.

Evaluación de Seguridad para Producción:

- **Puntos Fuertes:** Evita el riesgo común de subir secretos a repositorios Git públicos o privados. Permite actualizar credenciales en caliente y reiniciar contenedores con un clic.
- **Riesgos y Limitaciones:** Por defecto, los valores en PostgreSQL están guardados en texto claro o con codificación reversible si no se activa una clave maestra de cifrado en reposo (Encryption at Rest). Además, cualquier usuario con acceso al host puede inspeccionar las variables ejecutando `docker inspect` sobre el contenedor en ejecución. Para entornos de alta seguridad regulados (PCI-DSS, HIPAA), se recomienda complementar Dokploy con gestores de secretos externos dedicados como **HashiCorp Vault**, **AWS Secrets Manager** o el uso de **Docker Swarm Secrets** nativos.

5. ¿Qué estrategias de backup ofrece Dokploy para bases de datos y volúmenes persistentes?

Dokploy proporciona dos mecanismos principales de respaldo:

1. **Backups de Bases de Datos Lógicos (Dump-based):** Ejecuta periódicamente comandos nativos de volcado (`mysqldump` para MySQL, `pg_dump` para PostgreSQL, `mongodump` para MongoDB) dentro del contenedor. El archivo generado se comprime y se transfiere de forma segura a destinos de almacenamiento remoto compatibles con el protocolo **S3 API** (Amazon S3, MinIO, Cloudflare R2, Wasabi). Permite configurar expresiones `cron` (ej. `0 2 * * *` para las 2:00 AM) y políticas de retención (número máximo de copias a conservar antes de purgar).
2. **Backups de Volúmenes Persistentes (Volume Snapshots):** Dokploy cuenta con soporte para copiar los datos binarios de los volúmenes nombrados de Docker mediante contenedores auxiliares que empaquetan el contenido de `/var/lib/docker/volumes/...` en archivos tarball comprimidos.

Buenas prácticas: Se recomienda la regla de respaldo **3-2-1**: 3 copias de los datos, en 2 medios diferentes, con 1 copia almacenada fuera del sitio (Off-site en almacenamiento S3 externo inmutable con Object Lock).

6. ¿Cómo impacta el uso de un PaaS auto-hospedado en el consumo de recursos frente a Kubernetes o Docker Swarm puro?

El consumo de recursos de infraestructura varía drásticamente según la capa de abstracción seleccionada:

- **Docker Swarm Puro:** Presenta la sobrecarga más baja del mercado. El motor de Swarm está integrado en el daemon de Docker en lenguaje Go; requiere menos de **50 MB de memoria RAM** para mantener el estado del cluster y prácticamente cero CPU en reposo. Sin embargo, no ofrece interfaz web, gestión de secretos visual ni logs centralizados.
- **Dokploy (sobre Docker Swarm):** Añade una sobrecarga moderada y predecible. La suite completa de Dokploy (servidor Node.js, base de datos PostgreSQL interna y proxy Traefik v3) consume entre **250 MB y 450 MB de RAM** en reposo y menos del 1-2% de CPU. Esto lo hace viable incluso en VPS económicos de 1 vCPU y 1 GB de RAM.
- **Kubernetes (K8s / K3s):** Un cluster de Kubernetes estándar (etcd, kube-apiserver, kube-controller-manager, kube-scheduler, kubelet, CoreDNS, Flannel) consume como mínimo entre **1.5 GB y 3 GB de RAM** solo para el plano de control (Control Plane), incluso antes de desplegar la primera aplicación de usuario. Dokploy es entre **5 y 8 veces más ligero** que Kubernetes, lo que optimiza significativamente los costos en pequeñas y medianas infraestructuras.

7. En un entorno de producción real, ¿qué medidas adicionales de seguridad implementaría sobre esta instalación de Dokploy?

Para endurecer (hardening) una instalación de Dokploy orientada a producción empresarial, se deben implementar las siguientes capas de seguridad:

1. **Aislamiento de Red y Firewall (UFW / Security Groups):** Cerrar todos los puertos en el firewall perimetral excepto el 80 (HTTP), 443 (HTTPS) y un puerto SSH no estándar protegido con llave pública. El puerto administrativo 3000 de Dokploy *nunca debe estar expuesto a Internet público*; debe accederse exclusivamente a través de una red privada virtual (VPN WireGuard/Tailscale) o un túnel seguro con autenticación de identidad (Cloudflare Tunnels).
2. **Autenticación Multifactor (MFA/2FA):** Habilitar autenticación de doble factor obligatoria para todos los administradores en Dokploy y restringir el número de intentos fallidos de inicio de sesión (Fail2ban).
3. **Restricción del Docker Socket:** El montaje de `/var/run/docker.sock` otorga privilegios equivalentes a `root` sobre el host. Se recomienda implementar un proxy de socket como **docker-socket-proxy** para bloquear peticiones peligrosas de la API de Docker y limitar los verbos permitidos.
4. **Protección DDoS y WAF en el Edge:** Integrar Cloudflare u otro CDN con Web Application Firewall (WAF) frente a Traefik para filtrar ataques volumétricos, inyecciones SQL y mitigación de bots.
5. **Principio de Mínimo Privilegio en Contenedores:** Configurar los contenedores de las aplicaciones para ejecutarse como usuarios no privilegiados (`USER node` o `nobody`), con sistemas de archivos de raíz de solo lectura (`read-only root filesystem`) y `no-new-privileges` .
6. **Actualizaciones Desatendidas y Auditoría:** Activar `unattended-upgrades` en el host Linux para parches del kernel y exportar los logs de auditoría de Dokploy a un SIEM externo.

8. ¿Qué limitaciones identifica en Dokploy para entornos empresariales de alta disponibilidad?

Aunque Dokploy es una solución extraordinaria para entornos de desarrollo, staging, laboratorios y arquitecturas de microservicios de pequeña/mediana escala, presenta limitaciones claras frente a sistemas orquestadores de grado empresarial:

- **Punto Único de Fallo en el Plano de Control (Single Point of Failure - SPoF):** Aunque Dokploy puede gestionar servidores remotos y desplegar servicios en Docker Swarm multi-nodo, el dashboard central y su base de datos PostgreSQL residen por defecto en un único nodo primario. Si ese servidor colapsa, la interfaz administrativa y el motor de compilación quedan inoperables.
- **Almacenamiento Persistente Multi-Nodo Limitado:** En clusters multi-nodo, los volúmenes de Docker no se replican automáticamente entre servidores físicos a menos que se configure un sistema de almacenamiento distribuido externo complejo (NFS, Ceph, GlusterFS o Longhorn), lo cual no viene empaquetado de forma nativa en Dokploy.
- **Métricas y Alertas Avanzadas:** El monitoreo interno es adecuado para diagnóstico básico, pero carece de la riqueza analítica, trazabilidad distribuida (OpenTelemetry, Jaeger) y retención histórica de métricas que ofrecen stacks dedicados como **Prometheus + Grafana**.
- **Ausencia de Autoscaling Dinámico Horizontal:** Dokploy no cuenta con un escalador automático de pods/tareas basado en métricas en tiempo real (como el Horizontal Pod Autoscaler - HPA de Kubernetes), requiriendo intervención manual para aumentar el número de réplicas durante picos imprevistos de tráfico.

11. Entregable 3: Informe de Resultados

A continuación se consolidan los enlaces de acceso público/local de los servicios desplegados, las cadenas de conexión parametrizadas de las bases de datos (con las credenciales de seguridad debidamente sanitizadas conforme a los lineamientos de la práctica) y el resumen del estado operativo de los componentes.

11.1 URLs de las Aplicaciones y Servicios Desplegados

Servicio	Tipo	URL de Acceso / Endpoint	Estado Operativo
Aplicación Principal (v2.0.0)	PaaS Webapp (Nixpacks)	`http://webapp.127.0.0.1.traefik.me/`	**Activo (200 OK)**
Healthcheck Aplicación	Monitoreo de Salud	`http://webapp.127.0.0.1.traefik.me/health`	**Saludable (Uptime OK)**
Información de Entorno	Telemetría de Runtime	`http://webapp.127.0.0.1.traefik.me/info`	**Activo (JSON OK)**
Microservicio Compose	Docker Compose App	`http://localhost:3001/`	**Activo (200 OK)**
phpMyAdmin Compose	Gestor Web MySQL	`http://localhost:8080/`	**Activo (Login OK)**
Panel Dokploy Core	PaaS Control Plane	`http://localhost:3000/`	**Operacional v0.30.10**

11.2 Cadenas de Conexión de Bases de Datos (Sanitizadas)

Las siguientes cadenas de conexión representan la configuración de acceso a los motores de persistencia. Conforme a las directrices de seguridad de la práctica, las contraseñas reales han sido sustituidas por marcadores de posición seguros (`[PASSWORD]`):

> **1. MySQL 8.0 (db-uni-quindio)**
>
> **Conexión Externa / Host Local:**
> `mysql://admin_quindio:[PASSWORD]@127.0.0.1:3306/uni_quindio_db`
>
> **Conexión Interna de Red Docker (Service Mesh):**
> `mysql://admin_quindio:[PASSWORD]@uniquindio-dbuniquindio-t5ksjq:3306/uni_quindio_db`

2. PostgreSQL 16 (db-postgres-quindio)

Conexión Externa / Host Local:

```
postgresql://analyst_quindio:[PASSWORD]@127.0.0.1:5432/analytics_db
```

Conexión Interna de Red Docker (Service Mesh):

```
postgresql://analyst_quindio:[PASSWORD]@uniquindio-dbpostgresquindio-wuzyel:5432/analytics_db
```

3. Redis 7 (redis-cache-quindio)

Conexión Externa / Host Local:

```
redis://127.0.0.1:6379/0
```

Conexión Interna de Red Docker (Service Mesh):

```
redis://uniquindio-rediscachequindio-rqcdwf:6379/0
```

4. Stack Compose (MongoDB y MySQL Compose)

MongoDB NoSQL (Interna compose-net):

```
mongodb://admin_compose:[PASSWORD]@compose-mongo:27017/compose_db?authSource=admin
```

MySQL Relacional (Interna compose-net):

```
mysql://admin_compose:[PASSWORD]@compose-mysql:3306/compose_db
```

12. Entregable 4: Análisis Comparativo

Para fundamentar la toma de decisiones arquitectónicas en la selección de plataformas de cómputo, se presenta a continuación un análisis comparativo multidimensional entre tres paradigmas: **Docker manual (CLI/Compose directo)**, **PaaS auto-hospedado (Dokploy)** y **PaaS comercial en la nube (Heroku, Vercel, Render, AWS Elastic Beanstalk)**.

12.1 Matriz Comparativa de Características

Dimensión Evaluada	Docker CLI / Compose Manual	Dokploy (Self-Hosted PaaS)	Cloud PaaS Comercial (Heroku / Vercel)
Curva de Aprendizaje	Alta (requiere dominar comandos, redes, volúmenes, Traefik/Nginx, Certbot).	Baja / Media (interfaz web intuitiva orientada a proyectos y servicios).	Muy baja (despliegue con un clic o comandos push simplificados).
Control de Infraestructura	Total (acceso directo al host, kernel y sistema de archivos).	Total (el software corre sobre servidores propios, retención completa de datos).	Nulo / Restringido (caja negra administrada por el proveedor).
Gestión SSL / TLS	Manual o mediante sidecars complejos de Certbot.	Automatizada nativamente mediante Traefik v3 y Let's Encrypt.	Automatizada y gestionada por el proveedor.
Despliegue Continuo (CI/CD)	Requiere configurar GitHub Actions o GitLab CI con llaves SSH al host.	Nativo mediante webhooks automáticos con detección de ramas Git.	Nativo e integrado profundamente con GitHub/GitLab.
Bases de Datos Gestionadas	Manual (redacción de compose, scripts de backup y mapeo de volúmenes).	Aprovisionamiento en 1 clic con interfaz gráfica y backups S3.	Servicio gestionado de pago con costo incremental elevado.
Monitoreo y Métricas	Requiere instalar agentes externos (Prometheus, cAdvisor, Node Exporter).	Dashboard integrado en tiempo real (CPU, RAM, Disco, Red).	Paneles avanzados incluidos (con cargos adicionales por retención).
Escalabilidad	Manual o configurando clusters de Swarm / Kubernetes.	Soporte multi-nodo a través de Docker Swarm y gestión remota.	Elástica y automática (auto-scaling bajo demanda).
Vendor Lock-in	Nulo (contenedores Docker estándar portables a cualquier VPS).	Nulo (los contenedores y volúmenes siguen el estándar abierto OCI).	Medio / Alto (dependencia de add-ons, buildpacks propietarios y APIs).

12.2 Ventajas y Desventajas de un PaaS Auto-Hospedado (Dokploy)

Ventajas Competitivas

- **Soberanía y Privacidad de Datos:** La totalidad del código fuente, bases de datos y registros reside dentro de los límites de la infraestructura propia, cumpliendo con regulaciones estrictas de protección de datos (GDPR, Habeas Data).
- **Previsibilidad de Costos:** El costo de operación es plano y predecible; no existen sorpresas de facturación asociadas a transferencias de ancho de banda o cobros por número de peticiones.
- **Flexibilidad Ilimitada:** Permite hospedar cualquier tipo de contenedor, motor de base de datos o stack de software sin restricciones impuestas por el catálogo de un proveedor comercial.
- **Eficiencia de Recursos:** Permite consolidar múltiples proyectos, entornos de desarrollo y microservicios en un único servidor dedicado o VPS, aprovechando al máximo la memoria y CPU disponibles.

Desventajas y Consideraciones de Mantenimiento

- **Responsabilidad Operativa (Ops Overhead):** El equipo interno debe encargarse de la seguridad del sistema operativo base, parches del kernel, configuración de cortafuegos y rotación de certificados.
- **Sin SLA Comercial:** La recuperación ante desastres o contingencias depende de los procedimientos internos implementados por la organización, sin contar con el respaldo de un contrato de nivel de servicio garantizado por un tercero.
- **Gestión de Alta Disponibilidad Manual:** Lograr redundancia geográfica y tolerancia a fallos multi-región requiere diseñar y mantener manualmente la replicación de bases de datos y sistemas de archivos distribuidos.

12.3 Análisis de Costo-Beneficio (TCO y ROI)

Para ilustrar el impacto financiero, consideremos un caso representativo de una aplicación web compuesta por 3 microservicios (Frontend, Backend API, Worker) acompañados de MySQL, Redis y 50 GB de almacenamiento persistente con un tráfico mensual moderado (500,000 visitas/mes):

Concepto de Costo Mensual	Heroku / Render (Comercial)	AWS Elastic Beanstalk / ECS	Dokploy en VPS Dedicado (Hetzner / DigitalOcean)
Cómputo (3 Contenedores)	$75 USD (3 x $25 Standard Dynos)	$42 USD (t3.medium + ALB)	$14 USD (VPS 4 vCPU, 8 GB RAM)
Base de Datos MySQL Gestionada	$50 USD (Standard tier)	$35 USD (RDS db.t3.small)	$0 USD (Incluida en el VPS de Dokploy)
Redis en Memoria	$15 USD (Mini tier)	$18 USD (ElastiCache t3.micro)	$0 USD (Incluido en el VPS de Dokploy)
Transferencia de Datos (Bandwidth)	$20 USD (Excedentes de red)	$15 USD (Data transfer out)	$0 USD (20 TB incluidos en el proveedor VPS)
Total Mensual Estimado	**$160 USD / mes**	**$110 USD / mes**	**$14 USD / mes**
Costo Anual Total	**$1,920 USD / año**	**$1,320 USD / año**	**$168 USD / año**

Conclusión Financiera y de Infraestructura

El uso de un PaaS auto-hospedado como **Dokploy** representa un **ahorro económico de más del 87% anual** frente a proveedores PaaS tradicionales. Para universidades, centros de desarrollo, startups y departamentos de TI que cuentan con competencias operativas intermedias en Linux y Docker, Dokploy constituye la alternativa más eficiente, equilibrando la simplicidad de la experiencia de usuario (Developer Experience - DX) con la soberanía total y optimización financiera de la infraestructura.

13. Conclusiones Generales de la Práctica

1. **Eficiencia en el Despliegue de Infraestructura:** Dokploy demostró reducir drásticamente el tiempo requerido para poner en producción servicios contenerizados. Tareas que tradicionalmente demandan horas de configuración manual de proxies inversos, enrutamiento SSL y balanceo de carga se completaron en cuestión de minutos.
2. **Flexibilidad Arquitectónica:** La capacidad de alternar entre servicios individuales gestionados bajo Docker Swarm y stacks multi-servicio orquestados mediante Docker Compose permite a los administradores de infraestructura abordar tanto microservicios modernos como aplicaciones legadas (legacy) con mínimas modificaciones.
3. **Automatización DevOps (CI/CD):** La integración nativa con repositorios Git y el soporte para webhooks de auto-despliegue democratizan las prácticas modernas de entrega continua, permitiendo a los equipos de desarrollo iterar con confianza gracias a los despliegues progresivos sin tiempo de inactividad (Zero Downtime Rollouts).
4. **Observabilidad y Resiliencia Integradas:** La centralización de registros en vivo, la telemetría de consumo de recursos y los mecanismos de copias de seguridad programadas hacia almacenamiento S3 garantizan un gobierno integral de la plataforma alineado con los estándares de la gestión moderna de TI.